BD

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Bases de Dados

Ano Letivo de 2024/2025

Sistema de Base de Dados para a GreenMotion Solutions: Relatório Técnico

G1: Gonçalo Barbosa | Gonçalo Carmo | Nuno Rebelo | Marco Sèvegrand

12 de abril de 2026

Data de Receção	
Responsável	
Avalição	
Observações	

Sistema de Base de Dados para a GreenMotion Solutions: Relatório Técnico

G1: Gonçalo Barbosa | Gonçalo Carmo | Nuno Rebelo | Marco Sèvegrand

12 de abril de 2026

Dedicamos este trabalho ao Professor Orlando Belo, pela sua orientação e apoio inestimáveis

Resumo

O presente relatório descreve o desenvolvimento de um Sistema de Base de Dados (SBD) para a empresa GreenMotion Solutions, especializada no aluguer de veículos ecológicos. O sistema foi projetado para integrar informações dispersas em diferentes fontes (PostgreSQL, ficheiros CSV e JSON) em uma plataforma centralizada, assegurando maior eficiência, consistência e segurança dos dados. Este relatório detalha as etapas do levantamento de requisitos, modelação concetual e lógica, implementação e validação do sistema, com ênfase na resolução de desafios de redundância de dados e nas necessidades operacionais da empresa. O sistema implementado reflete uma solução robusta e escalável, capaz de suportar as futuras expansões da empresa.

Área de Aplicação: Desenho e arquitetura de Sistemas de Bases de Dados

Palavras-Chave: Sistema de Base de Dados, GreenMotion Solutions, Orlando Formoso, Mobilidade Sustentável, MySQL, Modelação Lógica

Índice

1 Definição do Sistema **1**
1.1 Contextualização . . . 1
1.2 Fundamentação . . . 1
1.3 Objetivos . . . 2
1.4 Viabilidade . . . 3
1.5 Recursos . . . 3
1.6 Equipa . . . 4
1.7 Plano de Execução . . . 4
1.8 Revisão e Aprovação do Projeto . . . 5

2 Definição dos Requisitos **7**
2.1 Definição do Método de Levantamento e Análise de Requisitos . . . 7
2.2 Levantamento dos Requisitos . . . 8
2.3 Análise e Organização dos Requisitos . . . 10
2.4 Validação dos Requisitos . . . 12

3 Modelação Concetual **13**
3.1 Apresentação da Abordagem de Modelação . . . 13
3.2 Identificação e Caracterização das Entidades . . . 13
3.2.1 Entidade *Veiculo* . . . 13
3.2.2 Entidade *Cliente* . . . 14
3.2.3 Entidade *PostoAluguer* . . . 14
3.2.4 Entidade *Aluguer* . . . 14
3.2.5 Entidade *Pagamento* . . . 14
3.3 Identificação e Caracterização dos Relacionamentos . . . 15
3.3.1 Relacionamento *gere* . . . 15
3.3.2 Relacionamento *participa* . . . 15
3.3.3 Relacionamento *solicita* . . . 16
3.3.4 Relacionamento *efetua* . . . 16
3.3.5 Relacionamento *envolve* . . . 16
3.4 Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos 17
3.4.1 Atributos de *Veiculo* . . . 17
3.4.2 Atributos de *PostoAluguer* . . . 17
3.4.3 Atributos de *Cliente* . . . 18
3.4.4 Atributos de *Aluguer* . . . 18
3.4.5 Atributos de *Pagamento* . . . 19

3.5 Apresentação e Explicação do Diagrama ER 19
3.6 Validação do Modelo Concetual 21

4 Modelação Lógica **23**
4.1 Construção do Modelo de Dados Lógico 23
4.2 Apresentação e Explicação do Modelo Lógico 23
4.3 Normalização de Dados 29
4.3.1 Primeira Forma Normal (1FN) 29
4.3.2 Segunda Forma Normal (2FN) 29
4.3.3 Terceira Forma Normal (3FN) 30
4.4 Validação do Modelo com Interrogações do Utilizador 30
4.4.1 Interrogação 1: Listar o histórico de aluguer de um cliente 30
4.4.2 Interrogação 2: Listar o cliente associado a um número de telemóvel . 31
4.4.3 Interrogação 3: Calcular a receita gerada por um posto de aluguer num dado intervalo de tempo 32
4.4.4 Interrogação 4: Calcular a taxa de ocupação de uma categoria de veículo numa dada data 32
4.4.5 Interrogação 5: Listar os veículos disponíveis para aluguer num dado intervalo de tempo 33

5 Implementação Física **35**
5.1 Apresentação e Explicação da Base de Dados Implementada 35
5.1.1 Criação da Base de Dados 35
5.1.2 Ordem de Criação das Tabelas 35
5.1.3 Explicação da Estrutura das Tabelas 36
5.2 Criação de utilizadores da base de dados 39
5.2.1 Estrutura de Papéis e Privilégios 39
5.2.2 Implementação dos Utilizadores 41
5.3 Povoamento inicial da base de dados 42
5.4 Cálculo do espaço da base de dados 44
5.4.1 Caracterização dos Tipos de Dados 44
5.4.2 Análise Teórica do Tamanho por Tabela 44
5.4.3 Análise Realista com a Arquitetura InnoDB 48
5.4.4 Análise Completa dos Índices 49
5.4.5 Dimensão Total e Overhead do Sistema 50
5.4.6 Projeções de Crescimento 50
5.4.7 Considerações e Recomendações 52
5.5 Definição e caracterização de vistas de utilização em SQL 52
5.6 Tradução das interrogações do utilizador para SQL 53
5.7 Indexação do Sistema de Dados 56
5.7.1 Visão Geral do Sistema de Indexação 56
5.7.2 Índices Propostos 57
5.7.3 Avaliação de Desempenho 57
5.8 Implementação de Procedimentos, Funções e Gatilhos 58
5.8.1 Transações 58

5.8.2 Procedimentos 59
5.8.3 Funções 59
5.8.4 Gatilhos 60
5.8.5 Considerações Finais 62

6 Implementação do Sistema de Migração Dados 63
6.1 Visão Geral do Sistema de Migração 63
6.2 Caracterização das Fontes de Dados 63
6.2.1 Base de Dados PostgreSQL (clientesbd) 63
6.2.2 Ficheiros CSV 63
6.2.3 Ficheiro JSON 64
6.3 Estratégia de Migração 64
6.3.1 Ordem de Migração 64
6.3.2 Mapeamento de Chaves Primárias 64
6.4 Tecnologias e Ferramentas Utilizadas 64
6.4.1 Linguagem de Programação: Python 64
6.4.2 Bibliotecas Especializadas 65
6.5 Implementação e Execução do Script 65
6.5.1 Funções Centrais de Processamento 65
6.5.2 Sequência de Migração e Processamento de Entidades 65
6.5.3 Transformações de Dados e Tratamento de Erros 66

7 Conclusões e Trabalho Futuro 67

Lista de Siglas e Acrónimos 70

Anexos 71
Anexo 1 - Tabela de Requisitos Levantados 71
Anexo 2 - Tabela de Requisitos de Descrição 74
Anexo 3 - Tabela de Requisitos de Manipulação 75
Anexo 4 - Tabela de Requisitos de Controlo 76
Anexo 5 - Código do ficheiro *schema.sql* 76
Anexo 6 - Código do ficheiro *privileges_setup.sql* 78
Anexo 7 - Código do ficheiro *privileges_setup.sql* 80
Anexo 8 - Código do ficheiro *import_script.py* 85
Anexo 9 - Código do ficheiro *procedures_functions_views.sql* 90
Anexo 10 - Código do ficheiro *triggers.sql* 95
Anexo 11 - Código do ficheiro *queries_call.sql* 96

Lista de Figuras

1.1 Diagrama de Gantt relativo ao plano de execução 5
1.2 Cabeçalho da ata de «Revisão e Aprovação do Projeto» 6

2.1 Cabeçalho da ata de «Levantamento de Requisitos» 8
2.2 Pre-visualização dos requisitos levantados 9
2.3 Pré-visualização dos requisitos de descrição 10
2.4 Pré-visualização dos requisitos de manipulação 11
2.5 Pré-visualização dos requisitos de controlo 11
2.6 Cabeçalho da ata de «Revisão e Validação dos Requisitos» 12

3.1 Diagrama ER após a primeira fase de construção 20
3.2 Versão final do diagrama ER 21
3.3 Cabeçalho da ata «Revisão e Aprovação do Modelo Concetual» 22

4.1 Resultado da conversão automática com o *brModelo* 24
4.2 Modelo lógico no *MySQL Workbench* 25
4.3 Primeira interrogação do utilizador 31
4.4 Segunda interrogação do utilizador 31
4.5 Terceira interrogação do utilizador 32
4.6 Quarta interrogação do utilizador 33
4.7 Quinta interrogação do utilizador 33

7.1 Diagrama de Gantt do Fluxo de Trabalho 68

Lista de Tabelas

3.1 Entidades 15
3.2 Relacionamentos 16
3.3 Atributos de Veiculo 17
3.4 Atributos de PostoAluguer 18
3.5 Atributos de Cliente 18
3.6 Atributos de Aluguer 19
3.7 Atributos de Pagamento 19

4.1 Dicionário de dados da relação "PostoAluguer". 26
4.2 Dicionário de dados da relação "Veiculo". 26
4.3 Dicionário de dados da relação "Cliente". 27
4.4 Dicionário de dados da relação "Telemovel". 27
4.5 Dicionário de dados da relação "Aluguer". 28
4.6 Dicionário de dados da relação "Pagamento". 29

5.1 Matriz de Privilégios 41
5.2 Espaço Ocupado pelos Tipos de Dados 44
5.3 Análise de um registo de PostoAluguer 45
5.4 Análise de um registo de Veiculo 45
5.5 Análise de um registo de Cliente 46
5.6 Análise de um registo de Telemovel 46
5.7 Análise de um registo de Aluguer 47
5.8 Análise de um registo de Pagamento 47
5.9 Análise de um registo de Pagamento 48
5.10 Análise de um registo de Pagamento 48
5.11 Análise de um registo de Pagamento 49
5.12 Análise do espaço dos Índices 49
5.13 Análise da Dimensao Total 50
5.14 Caption 51
5.15 Espaço ocupado pelos dados após 5 anos 51
5.16 Análise do espaço dos Índices 51

1 Definição do Sistema

1.1 Contextualização

A empresa *GreenMotion Solutions* foi fundada em Fevereiro de 2019 por Cristiano Ronaldo e está sediada em Lisboa, Portugal. Especializada no aluguer de veículos ecológicos, a empresa tem como missão facilitar a mobilidade verde, oferecendo soluções de transporte que utilizam fontes de energia alternativas, tanto a particulares como empresas, promovendo um futuro mais limpo. A *GreenMotion Solutions* visa tornar-se a principal fornecedora de soluções de mobilidade sustentável, liderando a transição para um mundo com emissões zero.

A *GreenMotion Solutions* apresenta uma ampla variedade de soluções de transporte asseguradas por uma frota diversificada de veículos elétricos, híbridos e movidos a hidrogénio de diferentes marcas e modelos, pronta para satisfazer as mais diversas necessidades. Atualmente, a *GreenMotion Solutions* possui um escritório central localizado na Avenida da Liberdade, em Lisboa, responsável pela supervisão das operações em três postos de aluguer, distribuídos pelo país. Estas unidades encontram-se estrategicamente situadas em Lisboa, Porto e Braga. A empresa conta com um total de 21 colaboradores, desde a diretora de operações, Cristina Ferreira, ao responsável pela componente tecnológica, Engº Orlando Formoso.

1.2 Fundamentação

Nos últimos tempos, a *GreenMotion Solutions* tem enfrentado um crescimento substancial, potenciado pelo aumento na demanda por veículos elétricos, sobretudo por parte de famílias e indivíduos conscientes do seu impacto ambiental. Este crescimento levou à necessidade de expandir significativamente a frota de forma a suportar as suas operações, no entanto, esta expansão revelou desafios significativos na gestão dos alugueres.

Atualmente, a empresa utiliza três sistemas heterogéneos (PostgreSQL, ficheiros CSV e JSON) em cada posto de aluguer, o que dificulta a centralização e a integração da informação. Esta fragmentação tem resultado em ineficiências operacionais, erros humanos e duplicações de dados, comprometendo a qualidade, a precisão e a segurança dos mesmos.

Com a finalidade de melhorar a eficiência dos seus processos de aluguer e dar resposta à procura crescente por soluções de mobilidade verde, Cristiano Ronaldo, atual CEO da *GreenMotion Solutions*, desafiou o Engº Orlando Formoso, responsável pela componente tecnológica da

empresa, a implementar um Sistema de Base de Dados. Neste sentido, Orlando Formoso subcontratou a *NexusLabs*, uma empresa líder no setor de desenvolvimento de software e soluções tecnológicas, que destacou uma equipa de quatro especialistas para o desenvolvimento do SBD da *GreenMotion Solutions*.

A implementação deste sistema representa um passo crucial na modernização dos processos da *GreenMotion Solutions*, alinhando a empresa com as melhores práticas do mercado, e preparando-a para enfrentar os desafios futuros com maior confiança e eficiência.

1.3 Objetivos

Para enfrentar os desafios atuais e preparar-se para o futuro, a *GreenMotion Solutions* estabeleceu uma série de objetivos estratégicos com a implementação do novo Sistema de Gestão de Base de Dados (SBD). Sob a liderança do Engº Orlando Formoso e da diretora de operações Cristina Ferreira, foram definidos os seguintes objetivos:

- Integrar todos os dados dispersos nos diferentes sistemas (PostgreSQL, ficheiros CSV e JSON) num único SBD, garantindo que todos os departamentos tenham acesso à informação atualizada e consistente.
- Armazenar de um modo estruturado os dados sobre os veículos, clientes e alugueres de forma a diminuir o risco de perda de informação e o tempo gasto na gestão desses dados.
- Sistematizar a gestão de alugueres, reduzindo tarefas manuais e garantindo maior eficiência nos processos administrativos.
- Assegurar um controlo rigoroso sobre a frota, permitindo a monitorização em tempo real da disponibilidade e estado dos veículos.
- Melhorar a experiência dos clientes, garantindo um serviço mais rápido, transparente e livre de erros, desde a reserva até à devolução do veículo.
- Permitir uma análise detalhada das reservas, ocupação da frota e padrões de utilização através de um *dashboard* personalizado em MS Power BI, contribuindo para um planeamento estratégico mais eficiente e informado.
- Desenvolver um sistema que possa crescer com a empresa, suportando a expansão futura e a adição de novos serviços.
- Garantir que o SBD seja flexível o suficiente para se adaptar a mudanças nos requisitos do negócio.
- Garantir que apenas utilizadores autorizados podem aceder a dados sensíveis e funcionalidades críticas.

Com a implementação do SBD, espera-se que a *GreenMotion Solutions* fortaleça a sua posição no setor da mobilidade sustentável, assegurando um crescimento estruturado e alinhado com as melhores práticas tecnológicas e operacionais.

1.4 Viabilidade

A *GreenMotion Solutions* acredita que, ao substituir a sua atual gestão por um Sistema de Gestão de Base de Dados, conseguirá reaver o retorno no investimento dentro de um período inferior a um ano. Assim, identificou como fatores de viabilidade os seguintes:

- Reduzir, já nos primeiros meses de funcionamento do sistema, o tempo gasto na gestão de reservas e contratos em aproximadamente 25%, permitindo lidar mais rapidamente com as necessidades de cada cliente e aumentando a eficiência operacional.
- Diminuir significativamente erros associados a reservas duplicadas e indisponibilidade de veículos em pelo menos 50%, evitando perdas financeiras e melhorando a experiência dos clientes.
- Diminuir em 12% as despesas com serviços externos (como os serviços de nuvem Google).

A implementação do SBD não é apenas uma necessidade operacional, mas uma estratégia essencial para o crescimento sustentável da *GreenMotion Solutions*, garantindo maior eficiência, segurança e competitividade no mercado da mobilidade sustentável.

1.5 Recursos

Para garantir o desenvolvimento e a manutenção do Sistema de Gestão de Base de Dados da *GreenMotion Solutions* de forma estável e eficiente, serão necessários recursos humanos e materiais adequados. Dentro dos recursos humanos contamos com duas equipas, a equipa interna e a equipa de desenvolvimento sendo estas constituídas, respetivamente, por:

- O Engº Orlando Formoso, responsável pela componente tecnológica da *GreenMotion Solutions*, que atuará como ponto de contacto principal entre a *GreenMotion Solutions* e a *NexusLab*, Cristina Ferreira, diretora de operações, que irá contribuir com o seu conhecimento das operações da empresa.
- Uma equipa de 4 especialistas, destacados pela *NexusLabs*, responsável pelo design, implementação e suporte técnico do sistema. É composta por um analista de dados, um arquiteto de bases de dados, um engenheiro de bases de dados e um administrador de bases de dados.

Na categoria de recursos materiais, consideram-se os recursos de hardware e recursos de software e licenças, nomeadamente:

- Servidor local para hospedar a base de dados e computadores adequados para a gestão e operação do sistema.
- Ferramenta para a elaboração da modelação concetual (*BrModelo*).
- Ferramenta para o desenvolvimento do SBD (*MySQL WorkBench*).
- Ferramenta para desenvolvimento do *dashboard* (*MS Power BI*).
- Licença para o SBD (MySQL Standard Edition Subscription - On-Premises, 1-4 socket server).

1.6 Equipa

A equipa responsável pelo desenvolvimento do Sistema de Base de Dados da *GreenMotion Solutions* será liderada, a nível interno, pelo responsável de TI, o Engº Orlando Formoso, e por Cristina Ferreira, diretora de operações, cujo conhecimento das operações da empresa será crucial.

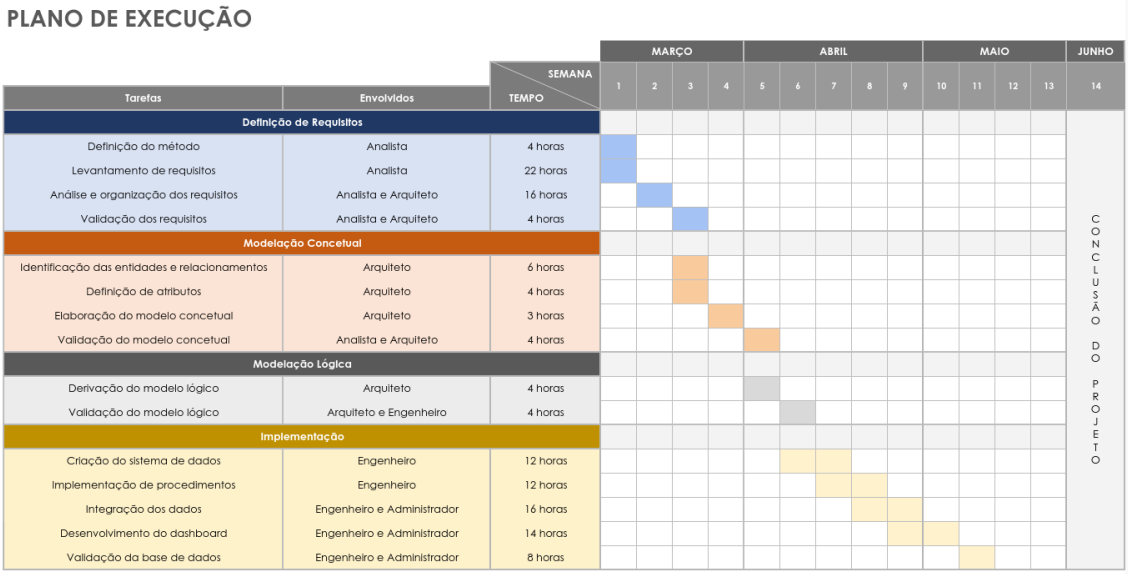
Já a nível externo, a equipa destacada pela *NexusLabs*, incumbida de implementar o Sistema de Base de Dados para a *GreenMotion Solutions*, é constituída por:

- Marco Sèvegrand, Analista de Dados, focado na análise e transformação de dados, levantamento de requisitos, e validação de esquemas e dados migrados.
- Gonçalo Barbosa, Arquiteto de Bases de Dados, responsável pela conceção e arquitetura do SBD, incluindo a definição de métodos e a elaboração de esquemas conceituais e lógicos.
- Nuno Rebelo, Engenheiro de Bases de Dados, especializado na implementação física do sistema, otimização de desempenho, e desenvolvimento do *dashboard* em *MS Power BI*.
- Gonçalo Carmo, Administrador de Bases de Dados, garantindo a integridade e segurança dos dados, liderando a integração dos dados na SBD.

1.7 Plano de Execução

Para garantir a conclusão do projeto dentro da janela de tempo estabelecida, de 3 de março a 30 de maio, elaborou-se um plano de execução detalhado que visa garantir a eficiência e a qualidade da realização do projeto. Este plano estrutura as diversas fases do projeto, desde a

definição dos requisitos até à implementação do sistema, assegurando que todos os objetivos estabelecidos sejam alcançados dentro do prazo estipulado.



PLANO DE EXECUÇÃO

Tarefas	Envolvidos	TEMPO \ SEMANA	MARÇO 1	2	3	4	ABRIL 5	6	7	8	9	MAIO 10	11	12	13	JUNHO 14
Definição de Requisitos																CONCLUSÃO DO PROJETO
Definição do método	Analista	4 horas														
Levantamento de requisitos	Analista	22 horas														
Análise e organização dos requisitos	Analista e Arquiteto	16 horas														
Validação dos requisitos	Analista e Arquiteto	4 horas														
Modelação Concetual																
Identificação das entidades e relacionamentos	Arquiteto	6 horas														
Definição de atributos	Arquiteto	4 horas														
Elaboração do modelo concetual	Arquiteto	3 horas														
Validação do modelo concetual	Analista e Arquiteto	4 horas														
Modelação Lógica																
Derivação do modelo lógico	Arquiteto	4 horas														
Validação do modelo lógico	Arquiteto e Engenheiro	4 horas														
Implementação																
Criação do sistema de dados	Engenheiro	12 horas														
Implementação de procedimentos	Engenheiro	12 horas														
Integração dos dados	Engenheiro e Administrador	16 horas														
Desenvolvimento do dashboard	Engenheiro e Administrador	14 horas														
Validação da base de dados	Engenheiro e Administrador	8 horas														

Figura 1.1: Diagrama de Gantt relativo ao plano de execução

A fase inicial do projeto envolve o levantamento de requisitos, análise e organização das necessidades de sistema de forma a garantir uma solução robusta e eficiente para a *GreenMotion Solutions*. O foco desta etapa será identificar as funcionalidades essenciais e assegurar que todas as necessidades sejam bem compreendidas e documentadas. Com os requisitos definidos e validados, será iniciada a modelação concetual do sistema, um passo importante na definição da estrutura e organização dos dados. A modelação lógica é a fase em que o esquema concetual será transformado num modelo lógico que pode ser implementado diretamente no sistema de gestão de bases de dados. Na fase de implementação, o sistema será fisicamente construído e configurado para atender aos requisitos definidos nas fases anteriores. Será desenvolvida a base de dados, os procedimentos e a *dashboard* necessários para a operação do sistema.

1.8 Revisão e Aprovação do Projeto

A revisão e aprovação do projeto ocorreu após a definição detalhada do sistema e elaboração do plano de execução. Durante esta etapa, as equipas da GreenMotion Solutions e da NexusLabs reuniram-se para analisar o documento de definição do sistema, os objetivos estabelecidos e o cronograma proposto. O Engº Orlando Formoso e Cristina Ferreira apresentaram as necessidades específicas da empresa aos especialistas da NexusLabs, que por sua vez expuseram a metodologia de desenvolvimento e as soluções técnicas previstas. Foram considerados os recursos disponíveis, os prazos estimados e o orçamento alocado. Com base nesta informação, a direção da GreenMotion Solutions aprovou oficialmente o início do projeto, autorizando a

alocação dos recursos necessários e estabelecendo os mecanismos de acompanhamento do progresso conforme o diagrama de Gantt previamente apresentado.

NEXUSLABS

GREEN MOTION SOLUTIONS

Revisão e Aprovação do Projeto

Data 11 / 03 / 2025

Horário: 11:00

1. OBJETIVO DA REUNIÃO
Análise e aprovação da proposta de execução, alocação de recursos e cronograma para o projeto de implementação do Sistema de Base de Dados da GreenMotion Solutions.

2. PARTICIPANTES PRESENTES	
NOME	POSIÇÃO
Eng° Orlando Formoso	**Responsável de TI (GreenMotion Solutions)**
Cristina Ferreira	**Diretora de Operações (GreenMotion Solutions)**
Nuno Rebelo	**Membro da equipa de desenvolvimento (NexusLabs)**
Marco Sèvegrand	**Membro da equipa de desenvolvimento (NexusLabs)**

Figura 1.2: Cabeçalho da ata de «Revisão e Aprovação do Projeto»

2 Definição dos Requisitos

2.1 Definição do Método de Levantamento e Análise de Requisitos

A definição precisa dos requisitos constitui uma etapa fundamental no desenvolvimento de um Sistema de Base de Dados (SBD). Para assegurar a recolha de informação detalhada e representativa da realidade operacional da *GreenMotion Solutions*, a equipa da *NexusLabs* adotou uma abordagem estruturada e multifacetada. A metodologia adotada foi desenhada para capturar não apenas as necessidades explícitas, mas também os requisitos implícitos e as expectativas dos utilizadores finais, sendo composta pelos métodos que se seguem.

Para o levantamento, foram realizadas entrevistas individuais semi-estruturadas com os principais *stakeholders* responsáveis pelas operações diárias da empresa, além de uma análise detalhada dos sistemas existentes, que permitiu identificar limitações, redundâncias e oportunidades de melhoria nos sistemas de informação atuais.

Na fase de análise, procedeu-se à decomposição de requisitos complexos em componentes mais concisas e tecnicamente precisas, facilitando a sua compreensão e posterior implementação. Foi também realizada uma análise de consistência e conflitos para identificar e resolver inconsistências, ambiguidades e conflitos entre os requisitos documentados, garantindo a coerência do conjunto final. Os requisitos foram organizados em documentação estruturada com três categorias principais: requisitos de descrição (definição dos dados a serem armazenados e suas características), requisitos de manipulação (especificação das operações a serem realizadas sobre os dados) e requisitos de controlo (determinação das regras de negócio, restrições e políticas de acesso aos dados).

Por fim, a validação envolveu a verificação dos requisitos documentados junto dos *stakeholders* através de reuniões de revisão, garantindo que refletem com precisão as necessidades da *GreenMotion Solutions*. A combinação destas técnicas visa obter um conjunto de requisitos abrangente, coerente e alinhado com as necessidades operacionais reais da empresa, minimizando o risco de omissões ou interpretações incorretas.

2.2 Levantamento dos Requisitos

O levantamento dos requisitos envolveu a identificação e documentação detalhada dos mesmos. Este processo foi conduzido de forma sistemática, seguindo uma sequência estruturada que garantiu a representatividade e a qualidade da informação recolhida. Foram conduzidas entrevistas estruturadas com os principais intervenientes no projeto, assegurando que todas as áreas relevantes da empresa estivessem representadas. Os participantes incluíram:

- Cristina Ferreira (Diretora de Operações)
- Daniela Rodrigues (Coordenadora de Gestão de Frota)

Cada entrevista foi conduzida seguindo um protocolo semi-estruturado, projetado para equilibrar a consistência com a flexibilidade necessária para explorar tópicos emergentes.

NEXUSLABS

GREEN MOTION SOLUTIONS

Levantamento de Requisitos

Data 14 / 03 / 2025

Horário: 10:00

1. OBJETIVO DA REUNIÃO
Reunião para identificação e documentação detalhada dos requisitos técnicos e operacionais necessários para o desenvolvimento do Sistema de Base de Dados da GreenMotion Solutions

2. PARTICIPANTES PRESENTES	
NOME	POSIÇÃO
Cristina Ferreira	Diretora de Operações (GreenMotion Solutions)
Daniela Rodrigues	Coordenadora de Gestão de Frota (GreenMotion Solutions)
Nuno Rebelo	Membro da equipa de desenvolvimento (NexusLabs)
Marco Sèvegrand	Membro da equipa de desenvolvimento (NexusLabs)

Figura 2.1: Cabeçalho da ata de «Levantamento de Requisitos»

O objetivo destas reuniões foi identificar os processos críticos da empresa, mapear os fluxos de trabalho e compreender as lacunas nos sistemas de informação atuais. Durante estas interações, foram levantadas questões como:

- Que dados são essenciais para a gestão dos alugueres?
- Que operações são necessárias fazer sobre os dados?
- Quais são os perfis de utilizadores e os seus níveis de acesso?

- Quais são os principais desafios enfrentados com os sistemas atuais?
- Que outras expectativas tem para o novo SBD?

As respostas obtidas permitiram delinear os requisitos funcionais e não funcionais do SBD, garantindo que o sistema atenderia às expectativas e necessidades da empresa.

A equipa de desenvolvimento teve acesso aos atuais sistemas de informação onde se encontravam os dados relativos ao aluguer de veículos. A análise destes sistemas possibilitou a identificação de redundâncias e limitações nos métodos atualmente utilizados para armazenar e processar dados. Foram detetados múltiplos pontos de ineficiência, incluindo:

- Duplicação de informação em diferentes sistemas e folhas de cálculo
- Inconsistências nos dados entre localizações
- Processos manuais propensos a erros em operações críticas
- Ausência de mecanismos robustos para garantir a integridade dos dados
- Limitações na capacidade de criação de relatórios analíticos

O processo de levantamento de requisitos possibilitou uma compreensão aprofundada das exigências operacionais da *GreenMotion Solutions*, permitindo definir um conjunto de requisitos para o desenvolvimento do novo SBD.

REQUISITOS LEVANTADOS

Requisitos

Identificador	Data	Hora	Descrição	Fonte	Contexto	Responsável
R-001	14/03/2025	10:00	Precisamos de um sistema para gerir o aluguer de veículos com diferentes características técnicas.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-002	14/03/2025	10:03	Cada veículo deve ter um código único, matrícula, categoria, marca e outras características importantes como tipo de motor e número de lugares.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
			...			
R-007	14/03/2025	10:42	Os pagamentos devem ser registados com valores, método de pagamento e data.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-008	14/03/2025	10:54	Cada veículo pertence a um único posto de aluguer, mas pode ser alugado várias vezes.	Cristina Ferreida	Entrevista	Marco Sèvegrand

Figura 2.2: Pre-visualização dos requisitos levantados

2.3 Análise e Organização dos Requisitos

Durante o levantamento de requisitos, a equipa de desenvolvimento, com base na análise da tabela global de requisitos, estruturou-os em três categorias conforme a sua função no sistema. Foram identificados: requisitos de descrição, relacionados com a estrutura e modelação da base de dados; requisitos de manipulação, focados nas operações de interação e modificação dos dados; e requisitos de controlo, destinados a gerir permissões e acessos por diferentes perfis de utilizadores. De seguida, apresentamos os requisitos principais de cada categoria, estando as listagens completas disponíveis nos anexos.

REQUISITOS DE DESCRIÇÃO

Req.Descrição

Identificador	Descrição	Revisor
RD-001	Um veículo é caracterizado por um código de identificação único (campo ID, gerado automaticamente), matrícula (Matricula), categoria (Categoria), marca (Marca), modelo (Modelo), tipo de motor (TipoMotor, como elétrico, híbrido...), número de lugares (NumLugares), autonomia (Autonomia), estado (Estado: Ativo ou Inativo) e tarifa diária (TarifaDiaria).	Marco Sèvegrand
RD-002	Um cliente é caracterizado por um código de idenficação único (numérico sequencial, exemplo: '00001'), o seu tipo (particular ou empresa), nome completo, número de de idenficação fiscal (NIF), carta de condução (número, validade e categoria), morada completa, pelo menos um número de telemóvel, email opcional e estado (ativo ou inativo).	Marco Sèvegrand
RD-003	Um posto de aluguer é caracterizado por um código de idenficação único (numérico sequencial, exemplo: '00001'), nome, morada completa, número de telefone, email e estado (ativo ou inativo).	Marco Sèvegrand
RD-004	Um aluguer é caracterizado por um um código de identificação único (numérico sequencial, exemplo: '00001'), data de início e fim e estado (agendado, iniciado, terminado ou cancelado).	Marco Sèvegrand
RD-005	Um pagamento é caracterizado por um código de identificação único (numérico sequencial, exemplo: '00001'), montante pago, método de pagamento, data de realização (que é nula enquanto o pagamento não for efetuado) e estado (pendente, realizado, cancelado, devolvido).	Marco Sèvegrand

Figura 2.3: Pré-visualização dos requisitos de descrição

REQUISITOS DE MANIPULAÇÃO		
Identificador	Descrição	Revisor
RM-001	Registar um novo veículo e atualizar as suas características.	Nuno Rebelo
RM-002	Registar um novo cliente e atualizar os seus dados.	Nuno Rebelo
...		
RM-008	Cancelar um aluguer e/ou pagamento.	Nuno Rebelo
RM-009	Listar os veículos disponíveis para aluguer num dado intervalo de tempo.	Marco Sèvegrand
RM-010	Listar os veículos com umas dadas características técnicas.	Marco Sèvegrand

Figura 2.4: Pré-visualização dos requisitos de manipulação

REQUISITOS DE CONTROLO		
Identificador	Descrição	Revisor
RC-001	Um veículo pode apenas estar alugado a um único cliente num dado instante.	Nuno Rebelo
RC-002	Um cliente particular não pode ter mais de 2 alugueres a decorrer em simultâneo.	Nuno Rebelo
RC-003	Apenas utilizadores com o papel de gestor podem registar, modificar e eliminar veículos.	Nuno Rebelo
RC-004	Apenas utilizadores com o papel de gestor podem registar, modificar e eliminar postos de aluguer, eliminar clientes, alugueres e pagamentos.	Nuno Rebelo
RC-005	Deve haver um Admin com acesso total e todos os privilégios.	Nuno Rebelo

Figura 2.5: Pré-visualização dos requisitos de controlo

Após uma revisão exaustiva do documento de requisitos já organizados, não foram encontradas inconsistências ou classificações erradas, possibilitando a equipa marcar uma reunião para a validação dos requisitos junto da *GreenMotion Solutions*.

2.4 Validação dos Requisitos

Após a revisão completa dos requisitos pela equipa de analistas, realizou-se uma reunião com todos os envolvidos com o objetivo de validar a informação e garantir que esta alinhada com os objetivos do projeto.

Durante o encontro, cada requisito, vertente e detalhe foi analisado minuciosamente pela equipa do projeto, sendo feitos ajustes e correções sempre que necessário para assegurar a clareza e exequibilidade das especificações.

Além disso, foram discutidas potenciais melhorias e antecipados possíveis desafios na implementação dos requisitos. No final, todos os participantes concordaram e aprovaram os requisitos definidos, garantindo um alinhamento comum para as próximas fases do projeto.

NEXUSLABS

GREEN MOTION SOLUTIONS

Revisão e Validação dos Requisitos

Data 18 / 03 / 2025

Horário: 15:30

1. OBJETIVO DA REUNIÃO
Análise detalhada e validação coletiva dos requisistos do SBD, com implementação de ajustes e identificação de desafios, resultando na aprovação final dos requisitos.

2. PARTICIPANTES PRESENTES	
NOME	POSIÇÃO
Cristina Ferreira	**Diretora de Operações (GreenMotion Solutions)**
Daniela Rodrigues	**Coordenadora de Gestão de Frota (GreenMotion Solutions)**
Eng° Orlando Formoso	**Responsável de TI (GreenMotion Solutions)**
Nuno Rebelo	**Membro da equipa de desenvolvimento (NexusLabs)**
Marco Sèvegrand	**Membro da equipa de desenvolvimento (NexusLabs)**

Figura 2.6: Cabeçalho da ata de «Revisão e Validação dos Requisitos»

3 Modelação Concetual

3.1 Apresentação da Abordagem de Modelação

Na sequência do levantamento e da interpretação dos requisitos, a equipa responsável deu início à conceção da arquitetura da base de dados. O primeiro passo consistiu na elaboração de um diagrama Entidade-Relacionamento, ferramenta gráfica que ilustra as entidades de um sistema e as suas interligações. Para este fim, recorreu-se ao software *brModelo* (Cândido, 2020), que permite criar diagramas seguindo uma variante da notação Chen, ferramenta com a qual os elementos da equipa já estavam familiarizados. O processo iniciou-se pela determinação das entidades, seguida do estabelecimento das ligações entre si. Posteriormente, procederam à definição dos atributos das entidades e à especificação das características dos relacionamentos.

Convém mencionar o procedimento aplicado a todas as entidades, relacionamentos e atributos: as suas denominações foram formuladas eliminando espaços, sinais diacríticos e preposições, resultando em termos mais concisos. Este sistema de nomenclatura alinha-se com aquele adotado no modelo lógico que será apresentado adiante e no modelo físico a ser implementado futuramente.

3.2 Identificação e Caracterização das Entidades

A criação de uma base de dados exige que se identifiquem com rigor as entidades que a constituem. Estas entidades funcionam como elementos estruturantes da base de dados, sendo imperativo defini-las antes de proceder à identificação dos relacionamentos e atributos correspondentes. Uma entidade corresponde a um grupo de objetos que partilham as mesmas propriedades, objetos estes que existem independentemente, seja de forma física (tangível) ou apenas concetual (Connolly & Begg, 2015).

3.2.1 Entidade *Veiculo*

A entidade "Veiculo"foi identificada como componente fundamental na estrutura da base de dados, conforme evidenciado em RD-001. Esta entidade de natureza tangível refere-se aos automóveis disponíveis para aluguer, representando um elemento central do sistema. Os veículos existem independentemente e partilham um conjunto de propriedades específicas detalhadas

nos requisitos, tais como matrícula, marca, modelo, entre outras características técnicas. A sua existência autónoma e atributos distintivos justificam plenamente a sua definição como entidade.

3.2.2 Entidade *Cliente*

A segunda entidade identificada, logo em RD-002, é "Cliente", que representa os indivíduos ou organizações que utilizam os serviços de aluguer de veículos. Esta entidade tangível constitui um elemento essencial para o funcionamento do negócio. Os clientes existem de forma independente e possuem um conjunto de atributos partilhados.

3.2.3 Entidade *PostoAluguer*

A entidade "PostoAluguer", referida em RD-003, representa os locais físicos onde se processam os alugueres e devoluções de veículos. Esta entidade tangível é parte integrante da estrutura operacional do sistema. Cada posto existe independentemente, com características próprias descritas, incluindo localização geográfica. Esta entidade é distinta de um mero atributo de "Veiculo"ou "Aluguer", pois possui existência autónoma e características que vão além da simples associação com outras entidades.

3.2.4 Entidade *Aluguer*

A entidade "Aluguer", descrita pormenorizadamente em RD-004, representa a operação principal do negócio. Trata-se de uma entidade concetual que regista a utilização temporária de um veículo por um cliente. Cada instância de aluguer existe de forma independente e caracteriza-se por um conjunto de atributos específicos, nomeadamente datas e estado.

3.2.5 Entidade *Pagamento*

A entidade "Pagamento", referida em RD-005, foi também identificada como componente essencial do sistema de gestão de alugueres. Esta entidade conceptual regista a transação financeira associada a cada aluguer de veículo. Cada pagamento existe com pelo menos uma ocorrência para cada aluguer, com atributos próprios, tais como montante, data e método de pagamento.

Entidade	Descrição	Requisitos
Veiculo	Automóveis disponíveis para aluguer na empresa.	RD-001
Cliente	Indivíduos ou organizações que alugam os veículos.	RD-002
PostoAluguer	Locais físicos onde ocorrem os alugueres e os veículos sao armazenados.	RD-003
Aluguer	Operação que regista o uso temporário de um veículo por um cliente.	RD-004
Pagamento	Transação financeira associada a cada aluguer de veículo	RD-005

Tabela 3.1: Entidades

3.3 Identificação e Caracterização dos Relacionamentos

A identificação dos relacionamentos na base de dados foi conduzida mediante análise meticulosa dos requisitos do sistema. Após definição das entidades principais, foram estabelecidas as associações entre estas, determinando assim a estrutura relacional do sistema.

3.3.1 Relacionamento *gere*

O relacionamento "gere"foi identificado entre as entidades "PostoAluguer"e "Veículo", conforme evidenciado nos requisitos RD-006 e RD-007. A cardinalidade desta relação é de um-para-muitos (1:N), uma vez que cada veículo está associado obrigatoriamente a um único posto, enquanto um posto pode gerir diversos veículos simultaneamente. Quanto à participação, um posto de aluguer pode existir sem ter veículos atribuídos (participação parcial), porém todo veículo deve necessariamente estar associado a um posto específico (participação total).

3.3.2 Relacionamento *participa*

O relacionamento "participa"conecta as entidades "Veiculo"e "Aluguer", como detalhado em RD-009 e RD-010. Este relacionamento apresenta cardinalidade um-para-muitos (1:N), pois um veículo pode ser utilizado em diversos alugueres ao longo do tempo, enquanto cada aluguer está associado a exatamente um veículo. A participação do "Veículo"é parcial, dado que um veículo pode existir no sistema sem ter sido ainda alugado. Por outro lado, a participação de "Aluguer"é total, pois não pode haver um aluguer sem um veículo específico associado.

3.3.3 Relacionamento *solicita*

O relacionamento "solicita"liga as entidades "Cliente"e "Aluguer", como especificado em RD-008 e RD-011. Este relacionamento possui cardinalidade um-para-muitos (1:N), onde um cliente pode efetuar múltiplos alugueres, mas cada aluguer está vinculado a um único cliente. A participação de "Cliente"é parcial, já que um cliente pode estar registado sem ter realizado qualquer aluguer. Já a participação de "Aluguer"é total, pois todo aluguer deve estar associado a um cliente específico.

3.3.4 Relacionamento *efetua*

O relacionamento "efetua"estabelece a conexão entre "Cliente"e "Pagamento", com cardinalidade um-para-muitos (1:N). Um cliente pode realizar diversos pagamentos, enquanto cada pagamento está associado a um único cliente. A participação de "Cliente"é parcial, visto que um cliente pode existir no sistema sem ter efetuado pagamentos. A participação de "Pagamento"é total, sendo impossível existir um pagamento sem um cliente associado.

3.3.5 Relacionamento *envolve*

Por fim, o relacionamento "envolve"conecta as entidades "Aluguer"e "Pagamento", com cardinalidade um-para-muitos (1:N) . Um aluguer pode estar associado a múltiplos pagamentos (por exemplo, pagamentos parciais, adiantamentos ou mesmo cancelados), enquanto cada pagamento corresponde a um aluguer específico. A participação de "Aluguer"é parcial, pois pode existir um aluguer registado sem pagamento associado (por exemplo, em fase de reserva). Contudo, a participação de "Pagamento"é total, uma vez que todo pagamento deve corresponder necessariamente a um aluguer específico.

Relacionamento	Entidade	Cardinalidade	Participaçao	Requisitos
gere	PostoAluguer Veiculo	1:N	Parcial - Total	RD-006 RD-007
participa	Veiculo Aluguer	1:N	Parcial - Total	RD-009 RD-010
solicita	Cliente Aluguer	1:N	Parcial - Total	RD-008 RD-009
efetua	Cliente Pagamento	1:N	Parcial - Total	RD-008 RD-010
envolve	Aluguer Pagamento	1:N	Parcial - Total	RD-009 RD-010

Tabela 3.2: Relacionamentos

3.4 Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos

3.4.1 Atributos de *Veiculo*

Para a entidade "Veiculo", baseado no descrito em RD-001, o atributo "ID"foi estabelecido como identificador único, servindo como chave primária. Os outros atributos incluem: "Categoria", que classifica o tipo de veículo; "Marca"e "Modelo", referentes às especificações do fabricante; "TipoMotor", que indica a motorização; "NumLugares", que especifica a capacidade de passageiros; "Autonomia", que regista a distância máxima percorrível; "Estado", que indica o estado operacional do veículo; e "TarifaDiaria", que armazena o valor do aluguer por dia.

Atributo	Domínio	Opcional	Exemplo
ID	INT	N	16142361
Matricula	VARCHAR	N	12 HR 23
Categoria	VARCHAR	N	Sedan
Marca	VARCHAR	N	Porche
Modelo	VARCHAR	N	Boxter
TipoMotor	VARCHAR	N	Híbrido Plug-In
NumLugares	INT	N	4
Autonomia	INT	N	500
Estado	ENUM	N	Ativo
TarifaDiaria	DECIMAL	N	103.00

Tabela 3.3: Atributos de Veiculo

3.4.2 Atributos de *PostoAluguer*

A entidade "PostoAluguer"apresenta como identificador único o atributo "ID", que funciona como chave primária. Os atributos adicionais incluem: "Nome", que armazena a designação do posto; "Morada", que regista o endereço completo; "NumTelefone", para contacto telefónico; "Email", para comunicações eletrónicas; e "Estado", que indica a situação operacional do posto. Todos estes atributos são considerados simples e obrigatórios, refletindo a informação essencial para a gestão dos locais físicos de operação.

Atributo	Domínio	Opcional	Exemplo
ID	INT	N	101
Nome	VARCHAR	N	Posto Central Lisboa
Morada	VARCHAR	N	Av. Liberdade, 100, Lisboa
NumTelefone	INT	N	211234567
Email	VARCHAR	N	contato@aluguer.pt
Estado	ENUM	N	Ativo

Tabela 3.4: Atributos de PostoAluguer

3.4.3 Atributos de *Cliente*

A entidade "Cliente"possui o atributo "ID"como identificador único. Os seus atributos simples são: "Nome", que regista a identificação pessoal; "Tipo", que distingue o cliente entre particular e empresa; "NIF", para identificação fiscal; "Morada", para o endereço; "Email", para comunicação eletrónica; "NumCartaCond", que armazena o número da carta de condução; "CategoriaCartaCond", que regista a categoria de veículos que o cliente está habilitado a conduzir; "ValidadeCartaCond", que indica a data de expiração do documento; e "Estado", que reflete a situação do cliente no sistema.

Atributo	Domínio	Opcional	Exemplo
ID	INT	N	202
Nome	VARCHAR	N	João Silva
Tipo	ENUM	N	Particular
NIF	INT	N	123456789
Morada	VARCHAR	N	Rua das Flores, 25, Porto
Email	VARCHAR	S	joao@email.pt
NumCartaCond	INT	N	987654321
CategoriaCartaCond	VARCHAR	N	B
ValidadeCartaCond	DATE	N	2026-08-15
Estado	ENUM	N	Ativo

Tabela 3.5: Atributos de Cliente

3.4.4 Atributos de *Aluguer*

A entidade "Aluguer"tem como identificador único o atributo "ID". Os atributos adicionais incluem: "DataHoraInicio"e "DataHoraFim", que registam o período de utilização do veículo; e "Estado", que indica a situação atual do aluguer.

Atributo	Domínio	Opcional	Exemplo
ID	INT	N	501
DataHoraInicio	DATETIME	N	2025-04-01 10:00
DataHoraFim	DATETIME	N	2025-04-05 16:00
Estado	ENUM	N	Ativo

Tabela 3.6: Atributos de Aluguer

3.4.5 Atributos de *Pagamento*

Por fim, a entidade "Pagamento"apresenta o atributo "ID"como identificador único. Os atributos simples incluem: "Montante", que regista o valor pago; "Metodo", que indica a forma de pagamento; "DataHora", que marca o momento da transação; e "Estado", que reflete a situação do pagamento.

Atributo	Domínio	Nulo	Exemplo
ID	INT	N	801
Montante	VARCHAR	N	150.00
Metodo	VARCHAR	N	Cartão de Crédito
DataHora	DATETIME	N	2025-04-05 16:05
Estado	ENUM	N	Pendente

Tabela 3.7: Atributos de Pagamento

A definição dos tipos de dados para os atributos baseou-se em práticas consolidadas de modelação de bases de dados, considerando a natureza da informação a armazenar e os requisitos de precisão necessários. Utilizou-se INT para valores numéricos inteiros, VARCHAR para textos com tamanho variável, DECIMAL para valores monetários, DATETIME para registo temporal completo e ENUM para atributos com conjunto predefinido de valores possíveis.

3.5 Apresentação e Explicação do Diagrama ER

Com a identificação e caracterização detalhadas das entidades, relacionamentos e atributos para o sistema de gestão de aluguer de veículos finalizada, procedeu-se à elaboração do diagrama Entidade-Relacionamento (ER). Esta representação visual constitui uma etapa crucial no desenvolvimento da base de dados, permitindo visualizar a estrutura concetual completa do sistema antes da sua implementação física.

Para construir o diagrama ER, a equipa de desenvolvimento utilizou a ferramenta *brModelo*. A equipa adotou uma abordagem monovista, construindo o diagrama na íntegra de uma só

vez, sem segmentação por módulos ou fases separadas de implementação.

O processo de elaboração do diagrama ER seguiu uma metodologia estruturada em quatro etapas distintas. Na primeira etapa, todas as cinco entidades identificadas ("Veiculo", "Cliente", "PostoAluguer", "Aluguer"e "Pagamento") foram posicionadas estrategicamente na tela do *brModelo*, representadas através de retângulos conforme a notação padrão. Para cada entidade, adicionaram-se todos os atributos correspondentes, utilizando a simbologia específica do *brModelo*: pequenos círculos para atributos simples, com a cardinalidade para atributos multivalorados e ligados a outros círculos subordinados para atributos compostos. Os atributos identificadores (chaves primárias) tem o seu círculo preenchido com uma cor sólida, seguindo a convenção da notação. Durante esta fase, também foram especificados os domínios de cada atributo e identificados quais eram opcionais, representados por pequenos círculos com linha tracejada.

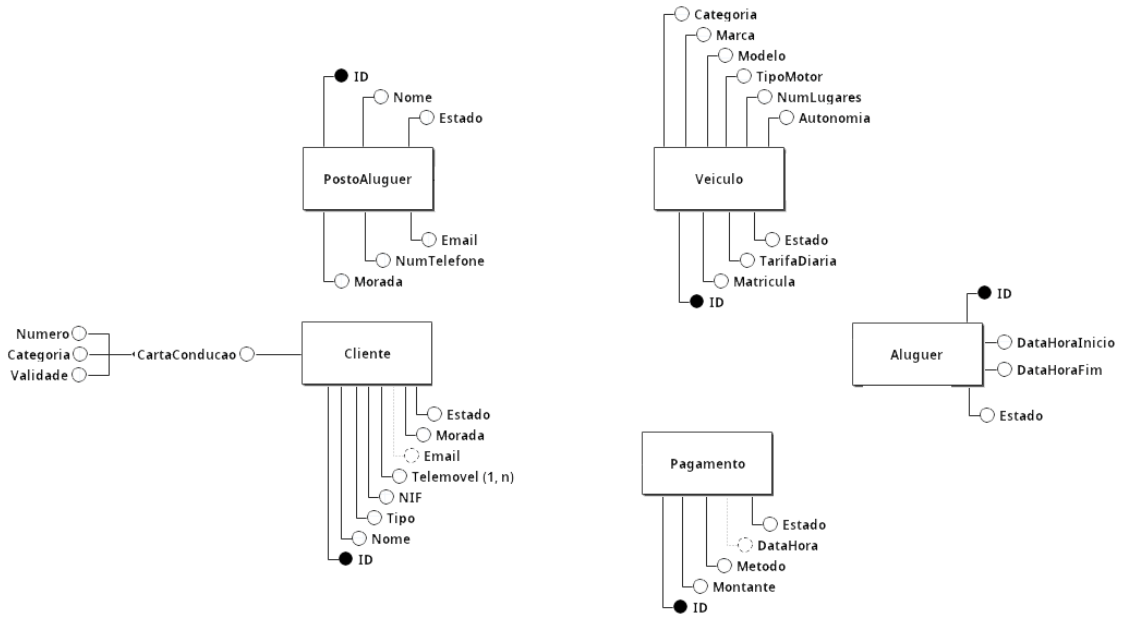


Figura 3.1: Diagrama ER após a primeira fase de construção

Na segunda etapa, estabeleceram-se os relacionamentos entre as entidades, representados por losangos que interligam as entidades participantes. A notação do *brModelo* para a cardinalidade e participação difere da notação de Chen, ao utilizar a notação Min/Max com cardinalidade *Look-Across* e participação *Look-Here*, uma representação onde em cada lado do relacionamento estas aparecem no formato (X, Y). Para cada relacionamento, definiram-se as cardinalidades mínimas (participação parcial ou total) e máximas (um ou muitos) de cada entidade envolvida. Nesta fase, verificou-se que não existia a necessidade de atributos nos relacionamentos para este modelo específico.

Na terceira etapa do processo, a equipa realizou uma verificação minuciosa dos requisitos originais do sistema, confrontando cada elemento do diagrama com as especificações docu-

mentadas. Este passo foi crucial para garantir que o modelo concetual não impediria nenhum dos requisitos de ser cumprido e que todas as regras de negócio estavam adequadamente representadas na estrutura do diagrama.

A quarta e última etapa consistiu no refinamento visual do diagrama, ajustando o posicionamento dos elementos gráficos para minimizar cruzamentos de linhas e maximizar a legibilidade. As entidades centrais ao negócio foram colocadas em posições estratégicas, e os relacionamentos foram dispostos de forma a facilitar a compreensão das interações do sistema.

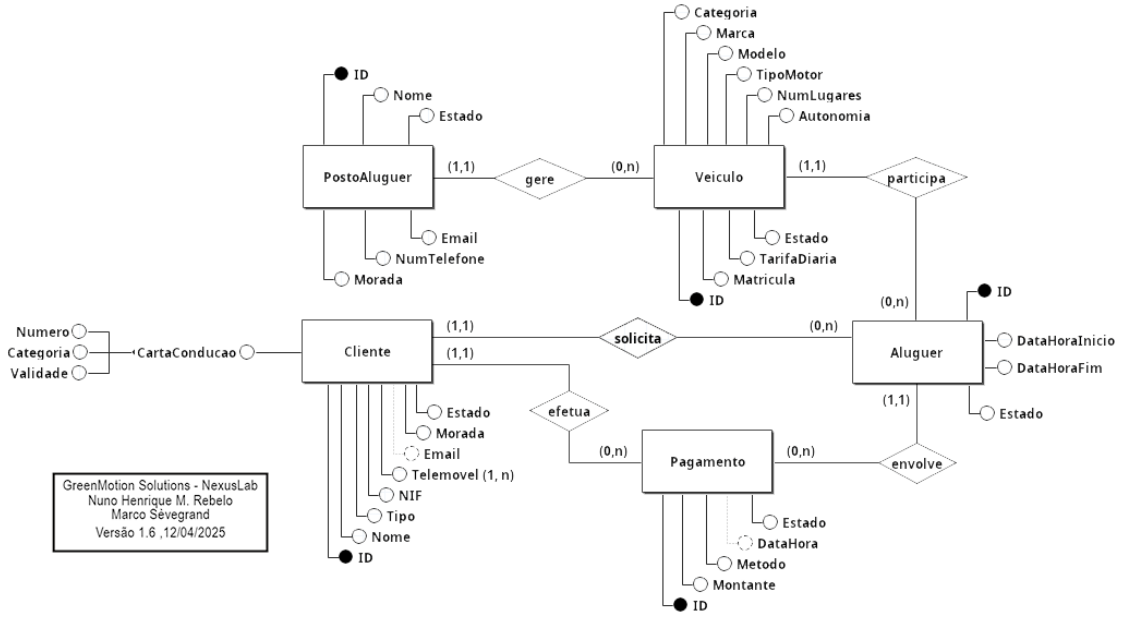


Figura 3.2: Versão final do diagrama ER

O resultado deste processo metodológico foi um diagrama ER completo e coeso, oferecendo uma representação visual clara da estrutura da base de dados para o sistema de aluguer de veículos. Este diagrama servirá como fundamento para a próxima fase do projeto: a transformação do modelo concetual em modelo lógico relacional.

3.6 Validação do Modelo Concetual

Após finalizada a construção do modelo concetual e da respetiva documentação, a equipa apresentou o trabalho ao Responsável de TI da *GreenMotion Solutions*, Orlando Formoso. Durante a revisão, foram ainda identificadas e corrigidas algumas incoerências. Posteriormente, foi também solicitado pelo responsável da *GreenMotion Solutions* um reforço na documentação, com maior ênfase na explicação do processo de dedução das entidades, relacionamentos e atributos uma vez que a equipa tinha uma documentação vaga, sem detalhar o raciocínio e as etapas seguidas para alcançar cada etapa.

NEXUSLABS

GREEN MOTION SOLUTIONS

Revisão e Aprovação do Modelo Concetual

Data 02 / 04 / 2025

Horário: 11:00

1. OBJETIVO DA REUNIÃO	
Revisão, correção e validação da documentação do modelo conceitual do Sistema de Base de Dados	
2. PARTICIPANTES PRESENTES	
NOME	**POSIÇÃO**
Eng° Orlando Formoso	Responsável de TI (GreenMotion Solutions)
Cristina Ferreira	Diretora de Operações (GreenMotion Solutions)
Nuno Rebelo	Membro da equipa de desenvolvimento (NexusLabs)
Marco Sèvegrand	Membro da equipa de desenvolvimento (NexusLabs)

Figura 3.3: Cabeçalho da ata «Revisão e Aprovação do Modelo Concetual»

4 Modelação Lógica

4.1 Construção do Modelo de Dados Lógico

Antes de avançar para a implementação física da base de dados, a equipa dedicou-se à construção e validação rigorosa do modelo lógico, assegurando que este refletia fielmente o diagrama ER completo já elaborado. Em vez de optar por uma abordagem faseada ou modular, foi decidido construir o modelo lógico na totalidade, o que permitiu garantir a coerência do sistema desde o início.

Para minimizar potenciais erros durante a transição do modelo concetual para o lógico, a equipa aplicou uma metodologia de construção redundante. Esta consistiu na realização de duas conversões distintas: uma manual, realizada por um dos elementos, e outra automática, recorrendo ao software *brModelo*. O *MySQL* Workbench foi utilizado na versão manual, já a pensar na futura integração com o SGBD *MySQL*. Após ambas as conversões, procedeu-se à comparação detalhada dos resultados, corrigindo-se eventuais discrepâncias identificadas.

A validação do modelo não se limitou à comparação das versões. Cada tabela foi analisada individualmente para garantir a conformidade com a Terceira Forma Normal (3FN), assegurando que as regras de normalização eram cumpridas. Adicionalmente, foram simuladas interrogações baseadas nos requisitos de manipulação, utilizando álgebra relacional, para verificar se o modelo respondia eficazmente às necessidades do sistema sem exigir alterações.

4.2 Apresentação e Explicação do Modelo Lógico

A etapa de construção manual do esquema lógico iniciou-se com a criação de cada tabela e a adição das colunas correspondentes. Na determinação dos colunas, foram considerados tanto os atributos simples quanto os compostos após a sua decomposição adequada. Para o atributo multivalorado, foi necessária a criação de uma nova tabela dedicada, que incluiu uma chave estrangeira referenciando a tabela à qual o atributo originalmente pertencia. A identificação das chaves primárias ocorreu sem esforço adicional, uma vez que o modelo concetual já definia explicitamente os identificadores únicos de cada entidade, os quais foram preservados nas tabelas correspondentes.

De seguida, procedeu-se à derivação dos relacionamentos, todos de natureza binária. Foi implementada uma abordagem sistemática baseada na cardinalidade, onde os relacionamentos

um-para-muitos resultaram na adição de uma chave estrangeira na tabela da entidade do lado "muitos", estabelecendo assim as ligações necessárias entre as tabelas relacionadas.

Com o modelo lógico construído manualmente concluído, a equipa avançou para a comparação com o modelo lógico obtido através da conversão automática do diagrama ER. Após uma análise detalhada, concluiu-se que os modelos eram essencialmente idênticos na sua estrutura e funcionalidade, diferindo apenas na representação da relação derivada do atributo multivalorado, devido às diferenças nas abordagens de derivação entre o processo manual e o algoritmo de conversão automática. Deste modo, optou-se por escolher o modelo lógico construido manualmente no *MySQL Workbench*.

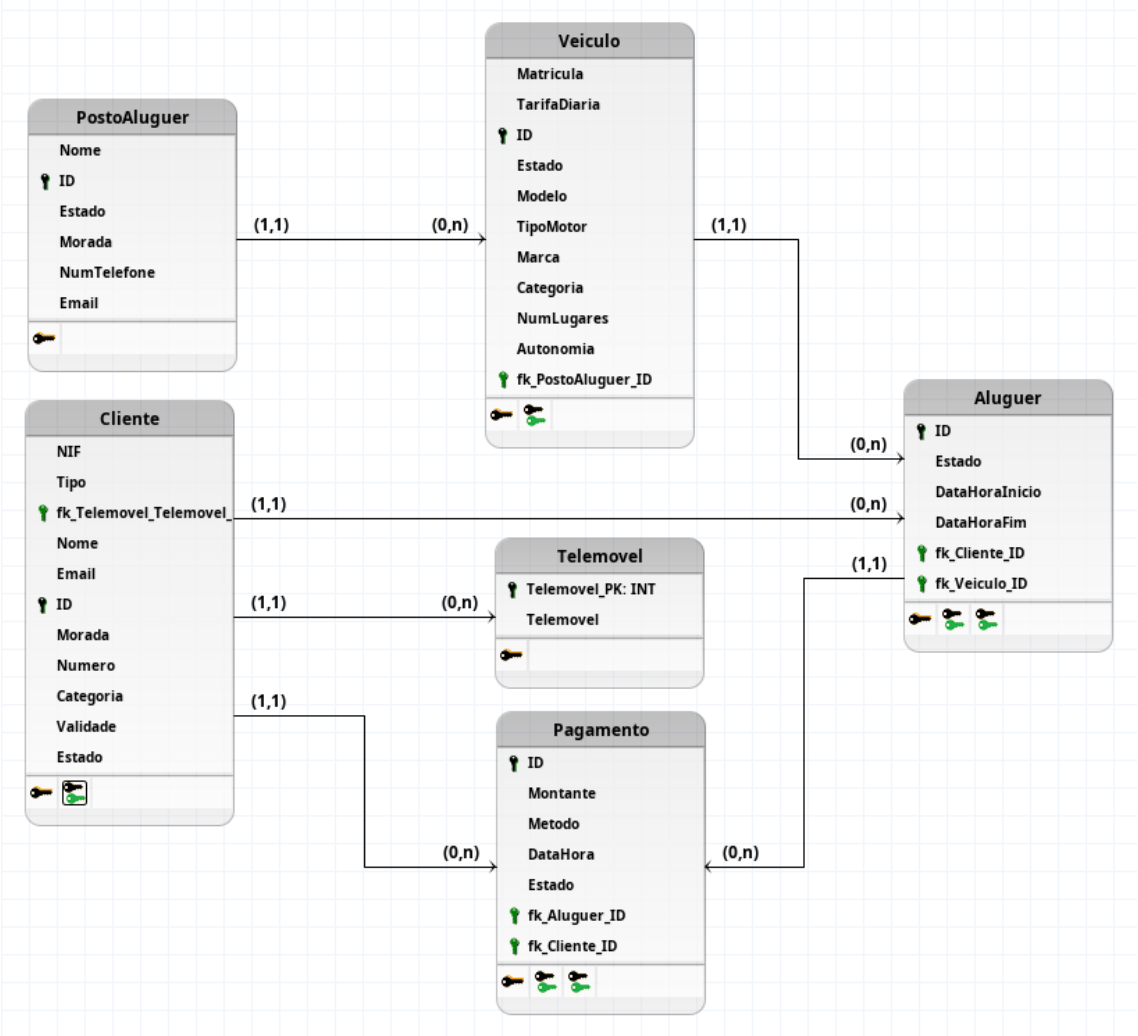


Figura 4.1: Resultado da conversão automática com o *brModelo*

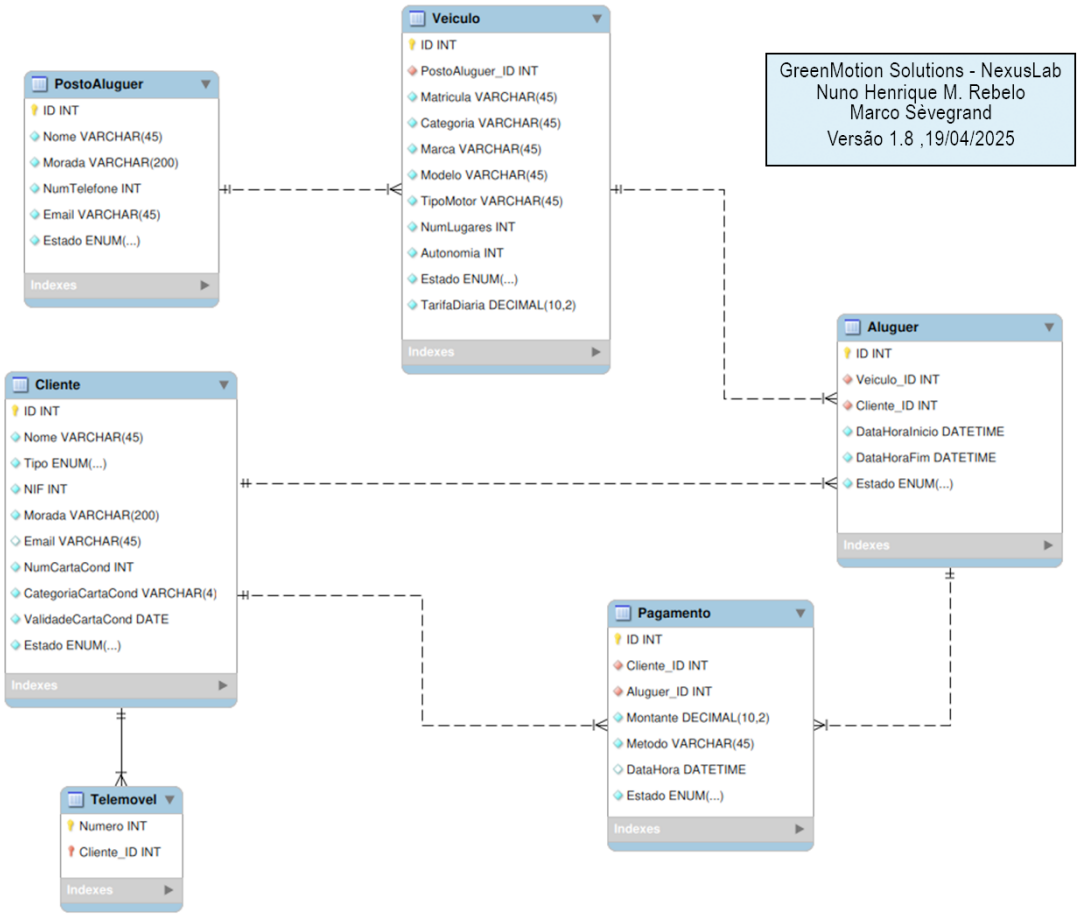


Figura 4.2: Modelo lógico no *MySQL Workbench*

Apresentamos, de seguida, o dicionário de dados, acompanhado da explicação do processo de construção de cada relação. Foi adotada a convenção de sublinhar os atributos que compõem a chave primária de cada relação, para facilitar a sua identificação.

A tabela "PostoAluguer" teve origem na entidade com o mesmo nome. Todos os seus atributos simples, "ID", "Nome", "Morada", "NumTelefone", "Email" e "Estado" deram origem a atributos diretos na relação. O atributo "Morada", embora com maior dimensão (até 200 caracteres), foi considerado simples e não composto, pelo que não sofreu decomposição adicional. O campo "Estado" foi modelado com recurso ao tipo ENUM, permitindo restringir os valores possíveis do atributo, representando estados pré-definidos do posto, nomeadamente "ativo"e "inativo". Não há atributos compostos nem multivalorados na entidade original, pelo que não foi necessário proceder a decomposições ou criações de tabelas adicionais. Determinou-se como chave primária o atributo "ID", um identificador único atribuído pela organização, garantindo total controlo sobre a sua unicidade e geração.

Atributo	Domínio	Nulo	Chave Estrangeira
ID	INT	Não	Não
Nome	VARCHAR(45)	Não	Não
Morada	VARCHAR(200)	Não	Não
NumTelefone	INT	Não	Não
Email	VARCHAR(45)	Não	Não
Estado	ENUM(...)	Não	Não

Tabela 4.1: Dicionário de dados da relação "PostoAluguer".

A tabela "Veiculo" tem origem na entidade do mesmo nome, cujos atributos simples são mapeados para atributos da relação ("ID", "Categoria", "Marca", "Modelo", "TipoMotor", "NumLugares", "Autonomia", "Estado" e "TarifaDiaria", todos de preenchimento obrigatório). Existe ainda um atributo resultante de um relacionamento de cardinalidade um-para-muitos, "PostoAluguer_ID", que representa a associação do veículo a um posto de aluguer. Devido à sua natureza de chave estrangeira, este atributo deve ter o mesmo domínio que a chave primária da tabela "PostoAluguer", sendo do tipo INT. Como a participação da entidade "Veiculo" neste relacionamento é total, o atributo "PostoAluguer_ID" não pode assumir valores nulos. Devido à sua unicidade, a única chave candidata (e consequentemente primária) é o atributo "ID".

Atributo	Domínio	Nulo	Chave Estrangeira
ID	INT	Não	Não
PostoAluguer_ID	INT	Não	Sim
Matricula	VARCHAR(45)	Não	Não
Categoria	VARCHAR(45)	Não	Não
Marca	VARCHAR(45)	Não	Não
Modelo	VARCHAR(45)	Não	Não
TipoMotor	VARCHAR(45)	Não	Não
NumLugares	INT	Não	Não
Autonomia	INT	Não	Não
Estado	ENUM(...)	Não	Não
TarifaDiaria	DECIMAL(10,2)	Não	Não

Tabela 4.2: Dicionário de dados da relação "Veiculo".

A tabela "Cliente" tem origem na entidade do mesmo nome, cujos atributos simples são mapeados para atributos da relação ("ID", "Nome", "Tipo", "NIF", "Morada", "Email" e "Estado"). Todos estes atributos, exceto o "Email", são obrigatórios, não havendo indicação de que algum deles possa assumir valores nulos. Os atributos "NumCartaCond", "CategoriaCartaCond" e "ValidadeCartaCond" formam-se a partir do atributo composto "Carta de Condução"presente no modelo conceptual.

O atributo "Tipo" é do tipo ENUM, representando diferentes categorias de cliente (por exemplo, particular, empresa, enquanto o atributo "Estado" também é um ENUM, utilizado para refletir a situação atual do cliente no sistema (ativo, inativo). Devido à sua unicidade e controlo interno, a única chave candidata (e consequentemente primária) é o atributo "ID".

Atributo	Domínio	Nulo	Chave Estrangeira
ID	INT	Não	Não
Nome	VARCHAR(45)	Não	Não
Tipo	ENUM(...)	Não	Não
NIF	INT	Não	Não
Morada	VARCHAR(200)	Não	Não
Email	VARCHAR(45)	Sim	Não
Estado	ENUM(...)	Não	Não
NumCartaCond	INT	Não	Não
CategoriaCartaCond	VARCHAR(4)	Não	Não
ValidadeCartaCond	DATE	Não	Não

Tabela 4.3: Dicionário de dados da relação "Cliente".

A tabela "Telemovel" foi derivada do atributo multivalorado de mesmo nome da entidade "Cliente". Cada registo nesta tabela representa um número de telemóvel associado a um cliente, pelo que contém dois atributos: "Numero" e "Cliente_ID". O atributo "Cliente_ID" é uma chave estrangeira que faz referência à tabela "Cliente", devendo por isso ter o mesmo domínio que a chave primária desta, ou seja, do tipo INT. Dado que o mesmo número pode, teoricamente, ser utilizado por diferentes clientes, e que cada cliente pode ter vários números associados, definiu-se como chave primária da tabela o par (Cliente_ID, Numero), formando assim uma chave composta que assegura a unicidade de cada número dentro do contexto de um cliente.

Como se trata de um atributo multivalorado, não faz sentido permitir valores nulos para o campo "Numero"; a ausência de um número deve simplesmente traduzir-se na inexistência de registo correspondente nesta tabela.

Atributo	Domínio	Nulo	Chave Estrangeira
Numero	INT	Não	Não
Cliente_ID	INT	Não	Sim

Tabela 4.4: Dicionário de dados da relação "Telemovel".

A tabela "Aluguer" tem origem na entidade do mesmo nome, cujos atributos simples foram mapeados para os seguintes atributos da relação: "ID", "DataHoraInicio", "DataHoraFim" e "Estado". O atributo "Estado" é do tipo ENUM, representando o estado atual do aluguer,

e os campos de data armazenam os momentos de início e fim do aluguer, sendo ambos do tipo DATETIME. A tabela inclui ainda dois atributos resultantes de relacionamentos de cardinalidade um-para-muitos: "Veiculo_ID" e "Cliente_ID", correspondendo respetivamente às ligações com as entidades "Veiculo" e "Cliente". Ambos são chaves estrangeiras e, como tal, devem ter o mesmo domínio que as chaves primárias das tabelas a que se referem, neste caso INT. Dado que um aluguer não pode existir sem um cliente e um veículo associados, a participação nos relacionamentos é total, pelo que ambos os atributos "Cliente_ID" e "Veiculo_VIN" são obrigatórios (não podendo assumir valores nulos). A única chave candidata (e consequentemente chave primária) é o atributo "ID", garantindo a unicidade de cada registo de aluguer.

Atributo	**Domínio**	**Nulo**	**Chave Estrangeira**
ID	INT	Não	Não
Cliente_ID	INT	Não	Sim
Veiculo_ID	INT	Não	Sim
DataHoraInicio	DATETIME	Não	Não
DataHoraFim	DATETIME	Não	Não
Estado	ENUM(...)	Não	Não

Tabela 4.5: Dicionário de dados da relação "Aluguer".

Finalmente a tabela "Pagamento" tem também esta origem na entidade do mesmo nome, cujos atributos simples foram mapeados para os seguintes atributos da relação: "ID", "Montante", "Metodo", "DataHora" e "Estado". O atributo "Montante" é do tipo DECIMAL(10,2), representando o valor monetário do pagamento, enquanto "Metodo" armazena a forma de pagamento utilizada (como cartão, numerário, transferência, etc.). "DataHora" regista o momento em que o pagamento foi efetuado, sendo do tipo DATETIME, e "Estado" é do tipo ENUM, refletindo o estado atual do pagamento. A tabela possui ainda dois atributos resultantes de relacionamentos de cardinalidade um-para-muitos com as tabelas "Cliente" e "Aluguer", respetivamente "Cliente_ID" e "Aluguer_ID". Estes são chaves estrangeiras, devendo ter o mesmo domínio que as chaves primárias das tabelas a que se referem, ou seja, do tipo INT. Como um pagamento deve estar sempre associado a um cliente e a um aluguer, presume-se participação total em ambos os relacionamentos, pelo que não são permitidos valores nulos nestes campos. A única chave candidata, e consequentemente primária, é o atributo "ID", garantindo a unicidade de cada registo de pagamento no sistema.

Atributo	Domínio	Nulo	Chave Estrangeira
ID	INT	Não	Não
Cliente_ID	INT	Não	Sim
Aluguer_ID	INT	Não	Sim
Montante	DECIMAL(10,2)	Não	Não
Metodo	VARCHAR(45)	Não	Não
DataHora	DATETIME	Sim	Não
Estado	ENUM(...)	Não	Não

Tabela 4.6: Dicionário de dados da relação "Pagamento".

4.3 Normalização de Dados

O modelo lógico apresentado encontra-se normalizado até à Terceira Forma Normal (3FN). Esta normalização foi realizada com o objetivo de garantir a eliminação de redundâncias, anomalias de atualização e manter a consistência dos dados.

4.3.1 Primeira Forma Normal (1FN)

O modelo encontra-se na Primeira Forma Normal porque todos os seus atributos são atómicos e indivisíveis. Durante a criação do modelo lógico, houve a preocupação de decompor adequadamente qualquer atributo que pudesse armazenar múltiplos valores. Esta atomicidade é fundamental para garantir que cada célula da tabela contenha apenas um valor único no domínio estabelecido para esse atributo, eliminando assim grupos repetitivos de dados. A separação de entidades como Cliente e Telemovel demonstra esta preocupação, permitindo que informações potencialmente multivaloradas sejam armazenadas em tabelas distintas com as devidas relações de integridade referencial.

4.3.2 Segunda Forma Normal (2FN)

Para estar na Segunda Forma Normal, além de satisfazer a 1FN, é necessário que todos os atributos não-chave dependam totalmente da chave primária e não apenas de parte dela. No modelo apresentado, isto é verificado de duas formas. Primeiro, as tabelas com chaves primárias simples (como Cliente, Veiculo, PostoAluguer, Aluguer e Pagamento) estão automaticamente na 2FN, pois a chave primária sendo um único atributo não permite dependências parciais. Segundo, nas tabelas com chaves compostas, como potencialmente a tabela Telemovel, os atributos não-chave (se existissem) dependeriam da totalidade da chave e não apenas de parte dela. A arquitetura do modelo evita intencionalmente situações onde atributos poderiam depender de apenas uma parte da chave primária, seguindo assim os princípios fundamentais

da normalização.

4.3.3 Terceira Forma Normal (3FN)

A Terceira Forma Normal exige que, além de estar na 2FN, não existam dependências transitivas, ou seja, nenhum atributo não-chave deve depender de outro atributo não-chave. No modelo apresentado, todos os atributos não-chave dependem exclusivamente da chave primária da sua tabela, e não de outros atributos da mesma tabela. Esta característica é fundamental para eliminar redundâncias e evitar anomalias de inserção, atualização e exclusão. O modelo foi projetado cuidadosamente para que informações relacionadas, mas funcionalmente independentes, estejam adequadamente separadas em tabelas distintas, estabelecendo relações através de chaves estrangeiras em vez de replicar dados. Essa separação lógica de entidades como Cliente, Veiculo, Aluguer e Pagamento permite que cada tabela armazene apenas os dados diretamente relacionados à sua entidade principal, mantendo assim a integridade referencial e eliminando dependências transitivas.

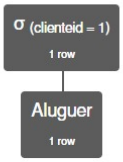
Em síntese, o modelo lógico apresentado demonstra um alto nível de normalização, respeitando as três primeiras Formas Normais, o que garante uma estrutura de dados robusta e livre de anomalias, fundamental para o desenvolvimento de um sistema de informação eficiente e confiável.

4.4 Validação do Modelo com Interrogações do Utilizador

Para confirmar que o modelo lógico proposto é adequado e atende às necessidades operacionais do sistema, realizou-se uma validação através de um conjunto de interrogações representativas derivadas dos requisitos de manipulação. Estas interrogações foram expressas em álgebra relacional, permitindo assim verificar se o modelo suporta eficazmente as operações essenciais do negócio.

4.4.1 Interrogação 1: Listar o histórico de aluguer de um cliente

A primeira interrogação implementada permite visualizar todos os alugueres associados a um cliente específico, identificado pelo seu ID. Esta funcionalidade é essencial para o atendimento personalizado e análise do histórico de relação com os clientes.



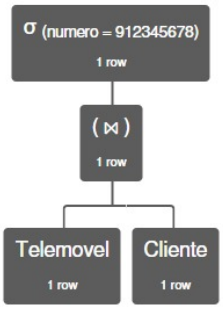
$\sigma_{(clienteid = 1)}$ (Aluguer)

Figura 4.3: Primeira interrogação do utilizador

Esta operação consiste numa seleção simples sobre a tabela Aluguer, filtrando os registos pelo ID do cliente. A consulta demonstra que o modelo possibilita facilmente o acesso ao histórico de alugueres, confirmando a eficácia da relação estabelecida entre as entidades Cliente e Aluguer.

4.4.2 Interrogação 2: Listar o cliente associado a um número de telemóvel

Esta interrogação permite identificar o cliente associado a um determinado número de telemóvel, funcionalidade frequentemente utilizada no atendimento telefónico e suporte ao cliente.



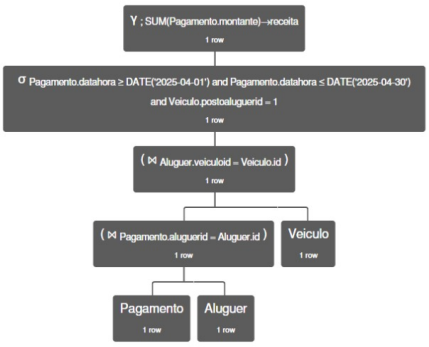
$\sigma_{(numero = 912345678)}$ (Telemovel ⋈ Cliente)

Figura 4.4: Segunda interrogação do utilizador

A consulta realiza uma junção natural entre as tabelas Telemovel e Cliente, seguida de uma seleção pelo número específico. Esta operação valida a eficácia da separação do atributo multivalorado "Telemovel"numa tabela própria, demonstrando como o modelo facilita a identificação de clientes a partir dos seus contactos.

4.4.3 Interrogação 3: Calcular a receita gerada por um posto de aluguer num dado intervalo de tempo

Esta interrogação mais complexa permite calcular a receita total gerada por um posto específico durante um determinado período, informação crucial para análises financeiras e de desempenho.



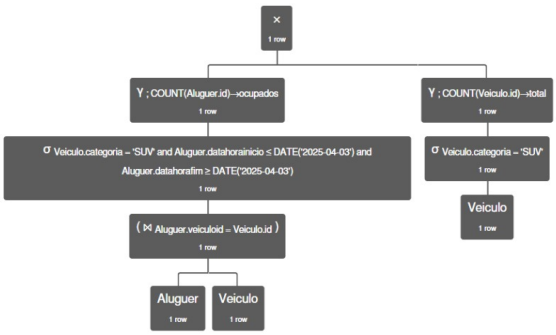
γ ; SUM(Pagamento.montante)→receita (σ Pagamento.datahora ≥ DATE('2025-04-01') and Pagamento.datahora ≤ DATE('2025-04-30') and Veiculo.postoaluguerid = 1 ((Pagamento ⋈ Pagamento.aluguerid = Aluguer.id Aluguer) ⋈ Aluguer.veiculoid = Veiculo.id Veiculo))

Figura 4.5: Terceira interrogação do utilizador

A consulta envolve junções entre as tabelas Pagamento, Aluguer e Veiculo, seguidas de uma seleção por data e posto de aluguer, e finalmente uma agregação para calcular a soma total. Esta operação valida que o modelo suporta adequadamente análises financeiras segmentadas por período e localização.

4.4.4 Interrogação 4: Calcular a taxa de ocupação de uma categoria de veículo numa dada data

Esta interrogação permite determinar a taxa de ocupação para uma categoria específica de veículos em determinada data, informação valiosa para gestão de frota e precificação dinâmica.



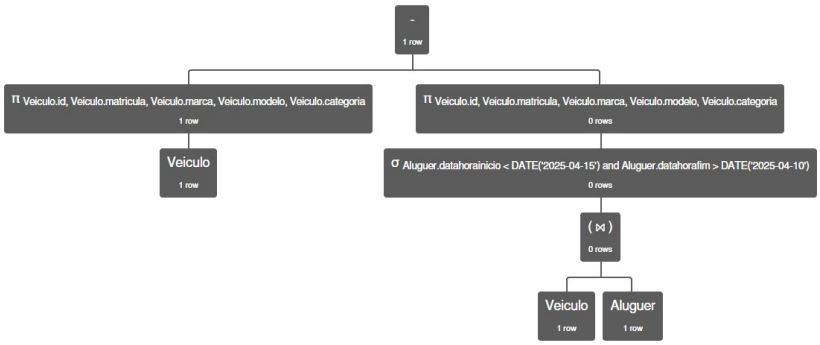
(γ ; COUNT(Aluguer.id)→ocupados (σ Veiculo.categoria = 'SUV' and Aluguer.datahorainicio ≤ DATE('2025-04-03') and Aluguer.datahorafim ≥ DATE('2025-04-03') (Aluguer ⋈ Aluguer.veiculoid = Veiculo.id Veiculo))) × (γ ; COUNT(Veiculo.id)→total (σ Veiculo.categoria = 'SUV' (Veiculo)))

Figura 4.6: Quarta interrogação do utilizador

Esta complexa operação envolve duas consultas separadas: uma para contar os veículos ocupados de uma categoria específica numa data, e outra para contar o total de veículos dessa categoria. O produto cartesiano dessas consultas permite calcular a taxa de ocupação. Esta interrogação valida a capacidade do modelo em suportar análises de disponibilidade e ocupação.

4.4.5 Interrogação 5: Listar os veículos disponíveis para aluguer num dado intervalo de tempo

Esta interrogação fundamental permite identificar quais veículos estão disponíveis para aluguer durante um intervalo de tempo específico, funcionalidade central para o negócio de aluguer de veículos.



π Veiculo.id, Veiculo.matricula, Veiculo.marca, Veiculo.modelo, Veiculo.categoria (Veiculo) - π Veiculo.id, Veiculo.matricula, Veiculo.marca, Veiculo.modelo, Veiculo.categoria (σ Aluguer.datahorainicio < DATE('2025-04-15') and Aluguer.datahorafim > DATE('2025-04-10') (Veiculo ⋈ Aluguer))

Figura 4.7: Quinta interrogação do utilizador

A consulta utiliza a operação de diferença entre o conjunto de todos os veículos e o conjunto dos veículos que têm alugueres sobrepostos com o período desejado. Esta operação confirma

que o modelo permite eficazmente identificar a disponibilidade de veículos, funcionalidade essencial para o sistema.

Através destas cinco interrogações representativas, foi possível confirmar que o modelo lógico suporta adequadamente as principais operações necessárias para o funcionamento eficaz do sistema de gestão de aluguer de veículos. A validação demonstrou que o modelo possibilita a extração de informação relevante para tomada de decisão, análise de desempenho e operações quotidianas do negócio.

5 Implementação Física

5.1 Apresentação e Explicação da Base de Dados Implementada

O modelo físico da base de dados GreenMotionDB foi desenvolvido através de um script SQL, a ser interpretado por um SGBD MySQL. Este script define, de forma sequencial e lógica, a criação da base de dados e de todas as tabelas necessárias para suportar o funcionamento de uma empresa de aluguer de veículos elétricos.

5.1.1 Criação da Base de Dados

O processo inicia-se com a criação da base de dados, garantindo que toda a informação será armazenada com suporte a múltiplos idiomas e símbolos, através da utilização do conjunto de caracteres *utf8mb4* e da colação `utf8mb4_unicode_ci`. Assim, é possível guardar nomes, moradas e outros campos textuais com acentuação e carateres especiais, sem perda de informação ou problemas de compatibilidade.

```
DROP DATABASE IF EXISTS GreenMotionDB;
CREATE DATABASE GreenMotionDB
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE GreenMotionDB;
```

A instrução `DROPDATABASEIFEXISTS` permite eliminar a base de dados, caso já exista, facilitando o desenvolvimento e a atualização do esquema. A diretiva USE GreenMotionDB indica ao SGBD que todas as operações seguintes serão realizadas nesta base de dados.

5.1.2 Ordem de Criação das Tabelas

A criação das tabelas segue uma ordem lógica, determinada pelas dependências de chaves estrangeiras entre as relações. Assim, garantimos que todas as referências são válidas no momento da criação:

1. PostoAluguer

2. Veiculo (depende de PostoAluguer)
3. Cliente
4. Telemovel (depende de Cliente)
5. Aluguer (depende de Veiculo e Cliente)
6. Pagamento (depende de Cliente e Aluguer)

5.1.3 Explicação da Estrutura das Tabelas

PostodeAluguer

A tabela PostoAluguer armazena os dados dos postos físicos onde os veículos podem ser alugados, incluindo nome, morada, número de telefone, email e o estado do posto. O campo ID é definido como INTAUTO_INCREMENTPRIMARYKEY, o que significa que o identificador do posto é gerado automaticamente pelo sistema à medida que novos registos são inseridos, garantindo unicidade e facilitando a gestão de chaves primárias. O campo Estado é do tipo ENUM e utiliza a restrição DEFAULT para assumir o valor 'Ativo' caso não seja especificado outro valor aquando da inserção de um novo registo, o que simplifica a criação de postos e previne estados indefinidos.

```
CREATE TABLE PostoAluguer(
    ID INT AUTO_INCREMENT PRIMARY KEY,
    Nome VARCHAR(45) NOT NULL UNIQUE,
    Morada VARCHAR(200) NOT NULL,
    NumTelefone INT UNSIGNED NOT NULL,
    Email VARCHAR(45) NOT NULL,
    Estado ENUM('Ativo','Inativo') NOT NULL DEFAULT 'Ativo'
);
```

Veiculo

A tabela Veiculo regista toda a informação relevante sobre cada veículo disponível para aluguer, como matrícula, categoria, marca, modelo, tipo de motor, número de lugares, autonomia, estado e tarifa diária. O campo ID utiliza também AUTO_INCREMENT, assegurando que cada veículo recebe um identificador único e gerado automaticamente. Cada veículo está associado a um posto de aluguer através da chave estrangeira PostoAluguer_ID. O campo Estado utiliza a restrição DEFAULT para assumir o valor 'Ativo' por omissão, facilitando a gestão dos veículos. A matrícula é definida como UNIQUE, impedindo a existência de dois veículos com a mesma matrícula, o que seria incoerente na realidade. Existem ainda uma restrição CHECK que impede tarifas diárias negativas.

```
CREATE TABLE Veiculo(
    ID INT AUTO_INCREMENT PRIMARY KEY,
    PostoAluguer_ID INT NOT NULL,
    Matricula VARCHAR(45) NOT NULL UNIQUE,
    Categoria VARCHAR(45) NOT NULL,
    Marca VARCHAR(45) NOT NULL,
    Modelo VARCHAR(45) NOT NULL,
    TipoMotor VARCHAR(45) NOT NULL,
    NumLugares INT UNSIGNED NOT NULL,
    Autonomia INT UNSIGNED NOT NULL,
    Estado ENUM('Ativo','Inativo') NOT NULL DEFAULT 'Ativo',
    TarifaDiaria DECIMAL(10,2) NOT NULL CHECK (TarifaDiaria >= 0),
    FOREIGN KEY (PostoAluguer_ID) REFERENCES PostoAluguer(ID)
);
```

Cliente

A tabela Cliente guarda a informação dos clientes, sejam eles individuais ou empresas, incluindo nome, tipo, NIF, morada, email, número e categoria da carta de condução, validade da carta e estado. O campo Estado utiliza `DEFAULT` para assumir o valor 'ativo' por defeito. O NIF e o número da carta de condução são ambos definidos como `UNIQUE`, assegurando que não existem duplicados destes dados críticos.

```
CREATE TABLE Cliente(
    ID INT AUTO_INCREMENT PRIMARY KEY,
    Nome VARCHAR(45) NOT NULL,
    Tipo ENUM('Individual','Empresa') NOT NULL,
    NIF INT NOT NULL UNIQUE,
    Morada VARCHAR(200) NOT NULL,
    Email VARCHAR(45),
    NumCartaCond INT NOT NULL UNIQUE,
    CategoriaCartaCond VARCHAR(4) NOT NULL,
    ValidadeCartaCond DATE NOT NULL,
    Estado ENUM('Ativo','Inativo') NOT NULL DEFAULT 'Ativo'
);
```

Telemovel

A tabela Telemovel permite associar múltiplos números de telemóvel a cada cliente, sendo composta por dois campos: Numero e Cliente_ID. A chave primária é composta por ambos os campos, garantindo que o mesmo número não pode ser associado duas vezes ao mesmo cliente. A chave estrangeira Cliente_ID referencia a tabela Cliente e utiliza `ONDELETECASCADE`, o que significa que, ao eliminar um cliente, todos os seus números de telemóvel são automaticamente removidos, evitando a existência de dados órfãos.

```
CREATE TABLE Telemovel(
    Numero INT NOT NULL,
    Cliente_ID INT NOT NULL,
    PRIMARY KEY (Numero, Cliente_ID),
    FOREIGN KEY (Cliente_ID)
        REFERENCES Cliente(ID)
        ON DELETE CASCADE
);
```

Aluguer

A tabela Aluguer regista cada aluguer realizado, associando um cliente a um veículo num determinado intervalo de tempo. Os campos DataHoraInicio e DataHoraFim definem o período do aluguer, e o campo Estado utiliza DEFAULT para assumir o valor 'agendado' por omissão. A restrição CHECK garante que a data de fim do aluguer é posterior à de início. As chaves estrangeiras Veiculo_ID e Cliente_ID utilizam ONDELETECASCADE. O estado do aluguer pode ser 'agendado', 'iniciado', 'concluído' ou 'cancelado'.

```
CREATE TABLE Aluguer(
    ID INT AUTO_INCREMENT PRIMARY KEY,
    Veiculo_ID INT NOT NULL,
    Cliente_ID INT NOT NULL,
    DataHoraInicio DATETIME NOT NULL,
    DataHoraFim DATETIME NOT NULL,
    Estado ENUM('Agendado','Iniciado','Concluído','Cancelado')
        NOT NULL DEFAULT 'Agendado',
    FOREIGN KEY (Veiculo_ID)
        REFERENCES Veiculo(ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Cliente_ID)
        REFERENCES Cliente(ID)
        ON DELETE CASCADE,
    CHECK (DataHoraFim > DataHoraInicio)
);
```

Pagamento

Por fim, a tabela Pagamento regista os pagamentos efetuados por cada aluguer, indicando o cliente, o aluguer correspondente, o montante, o método de pagamento, a data/hora e o estado do pagamento. O campo Estado utiliza DEFAULT para assumir o valor 'pendente' por defeito. A restrição CHECK em Montante impede valores negativos. Os estados possíveis para o pagamento são 'pendente', 'realizado', 'cancelado' e 'devolvido'.

```
CREATE TABLE Pagamento(
```

```
    ID INT AUTO_INCREMENT PRIMARY KEY,
    Cliente_ID INT NOT NULL,
    Aluguer_ID INT NOT NULL,
    Montante DECIMAL(10,2) NOT NULL CHECK (Montante >= 0),
    Metodo VARCHAR(45) NOT NULL,
    DataHora DATETIME,
    Estado ENUM ('Pendente','Realizado','Cancelado','Devolvido')
               NOT NULL DEFAULT 'Pendente',
    FOREIGN KEY (Cliente_ID)
        REFERENCES Cliente(ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Aluguer_ID)
        REFERENCES Aluguer(ID)
        ON DELETE CASCADE
);
```

5.2 Criação de utilizadores da base de dados

A base de dados GreenMotionDB foi concebida para ser utilizada por diversos utilizadores com diferentes níveis de acesso, conforme os requisitos de controlo especificados. Para garantir uma gestão eficiente e segura dos dados, optou-se por criar *roles* (papéis) que agrupam conjuntos de privilégios específicos, em vez de atribuir permissões individualmente a cada utilizador. Esta abordagem facilita a administração e a manutenção da base de dados, além de garantir que cada utilizador tenha apenas as permissões necessárias para desempenhar as suas funções.

5.2.1 Estrutura de Papéis e Privilégios

Antes de criar os papéis, foram analisados os requisitos de controlo para identificar as necessidades específicas de cada tipo de utilizador. Esta análise permitiu definir três papéis distintos: colaborador, gestor e admin. Cada um destes papéis foi atribuído com privilégios específicos, conforme as necessidades operacionais de cada função dentro da empresa. Esta estrutura foi implementada através do sistema de *roles* do MySQL, permitindo uma gestão centralizada dos privilégios:

```
CREATE ROLE 'colaborador', 'gestor', 'admin';
```

Colaborador

O papel de colaborador destina-se a funcionários que realizam operações do dia a dia. Estes utilizadores apenas podem consultar os dados dos postos de aluguer, garantindo que não alteram informações estruturais. No caso da tabela de veículos, o colaborador pode consultar

todos os dados e atualizar apenas o campo Estado, o que permite, por exemplo, marcar um veículo como disponível ou em manutenção, mas impede alterações a outros atributos críticos do veículo. Para a tabela de clientes, o colaborador pode consultar, inserir e atualizar dados, sendo-lhe vedada a eliminação de registos, o que evita perdas acidentais de informação. Já na tabela de telemóveis, o colaborador pode consultar, adicionar, atualizar e eliminar contactos, pois estes dados são frequentemente alterados. No caso dos alugueres, o colaborador pode consultar, inserir e atualizar apenas o campo Estado, permitindo-lhe controlar o progresso de cada aluguer sem acesso total à edição dos dados. Por fim, nos pagamentos, o colaborador pode consultar, inserir e atualizar os campos DataHora e Estado, o que permite registar e atualizar o estado dos pagamentos sem acesso a outros campos sensíveis.

```
GRANT SELECT ON PostoAluguer TO 'colaborador';
GRANT SELECT, UPDATE(Estado) ON Veiculo TO 'colaborador';
GRANT SELECT, INSERT, UPDATE ON Cliente TO 'colaborador';
GRANT SELECT, INSERT, UPDATE, DELETE ON Telemovel TO 'colaborador';
GRANT SELECT, INSERT, UPDATE(Estado) ON Aluguer TO 'colaborador';
GRANT SELECT, INSERT, UPDATE(DataHora, Estado) ON Pagamento TO
↪  'colaborador';
```

Gestor

O papel de gestor foi desenhado para utilizadores com responsabilidades de supervisão e controlo. O gestor tem privilégios completos de leitura, inserção, atualização e eliminação em todas as tabelas principais. Esta abrangência permite-lhe gerir toda a informação operacional, corrigir erros, remover dados obsoletos e garantir a qualidade dos dados, sendo essencial para funções de chefia e coordenação.

```
GRANT SELECT, INSERT, UPDATE, DELETE ON PostoAluguer TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Veiculo TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Cliente TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Telemovel TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Aluguer TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Pagamento TO 'gestor';
```

Administrador

O papel de admin é reservado a utilizadores com necessidades administrativas e de manutenção da base de dados. Estes utilizadores têm acesso total a todas as operações CRUD, podendo ainda gerir permissões, papéis e utilizadores.

```
GRANT ALL PRIVILEGES ON GreenMotionDB.* TO 'admin' WITH GRANT
↪  OPTION;
GRANT 'colaborador', 'gestor' TO 'admin' WITH ADMIN OPTION;
```

Matriz de Privilégios

CRUD (Create, Read, Update, Delete)

Tabela	Colaborador	Gestor	Administrador
PostoAluguer	R	CRUD	ALL PRIVILEGES
Veiculo	RU*	CRUD	ALL PRIVILEGES
Cliente	CRU	CRUD	ALL PRIVILEGES
Telemovel	CRUD	CRUD	ALL PRIVILEGES
Aluguer	CRU*	CRUD	ALL PRIVILEGES
Pagamento	CRU*	CRUD	ALL PRIVILEGES

* *Update* limitado a certas colunas da tabela

Tabela 5.1: Matriz de Privilégios

5.2.2 Implementação dos Utilizadores

No processo de criação e atribuição de utilizadores na base de dados GreenMotionDB, cada instrução SQL tem uma função específica e sequencial, permitindo configurar o acesso de forma controlada e eficiente.

```
CREATE USER 'colaborador1'@'localhost' IDENTIFIED BY 'password1';
CREATE USER 'colaborador2'@'localhost' IDENTIFIED BY 'password2';
CREATE USER 'gestor1'@'localhost' IDENTIFIED BY 'password3';
CREATE USER 'gestor2'@'localhost' IDENTIFIED BY 'password4';
CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'password5';
```

A instrução CREATEUSER serve para registar um novo utilizador na base de dados, definindo um nome de utilizador e uma palavra-passe. Por exemplo, ao executar `CREATEUSER'colaborador1'@'localhost'IDENTIFIEDBY'password1';`, o sistema cria um utilizador chamado colaborador1, que só pode aceder a partir do servidor local e cuja autenticação é feita pela palavra-passe indicada. Este comando é repetido para cada utilizador a criar, garantindo que cada um tem credenciais próprias e independentes.

```
GRANT 'colaborador' TO 'colaborador1'@'localhost';
GRANT 'colaborador' TO 'colaborador2'@'localhost';
GRANT 'gestor' TO 'gestor1'@'localhost';
GRANT 'gestor' TO 'gestor2'@'localhost';
GRANT 'admin' TO 'admin1'@'localhost';
```

Depois de criados, os utilizadores não têm privilégios por defeito, sendo necessário atribuir-lhes permissões através de papéis (roles). A instrução `GRANT'colaborador'TO'colaborador1'@'localhost';` associa o utilizador ao papel colaborador, herdando automaticamente todos os privilégios definidos para esse papel. Este método é preferível à atribuição

manual de permissões individuais, pois centraliza a gestão dos acessos e facilita futuras alterações. Se for necessário alterar os privilégios de um grupo de utilizadores, basta modificar o papel correspondente, e todos os utilizadores associados herdarão imediatamente as novas permissões.

```
SET DEFAULT ROLE 'colaborador' TO 'colaborador1'@'localhost';
SET DEFAULT ROLE 'colaborador' TO 'colaborador2'@'localhost';
SET DEFAULT ROLE 'gestor' TO 'gestor1'@'localhost';
SET DEFAULT ROLE 'gestor' TO 'gestor2'@'localhost';
SET DEFAULT ROLE 'admin' TO 'admin1'@'localhost';
```

A definição do papel padrão com `SETDEFAULTROLE'colaborador'TO'colaborador1'@'localhost';` garante que, ao iniciar sessão, o utilizador assume automaticamente o papel pretendido, sem necessidade de ativação manual. Isto evita situações em que um utilizador possa aceder com menos privilégios do que o necessário ou, inversamente, com mais permissões do que o previsto. O comando `SETDEFAULTROLE` é executado para cada utilizador, alinhando o seu perfil de acesso com as funções que irá desempenhar na organização.

Este conjunto de instruções permite estruturar de forma clara e modular o acesso à base de dados, facilitando a administração e a manutenção dos privilégios, bem como a integração de novos utilizadores ou a alteração de funções existentes.

5.3 Povoamento inicial da base de dados

O povoamento inicial da base de dados GreenMotionDB foi realizado com o objetivo de criar um conjunto de dados representativo que permita testar e validar o funcionamento do sistema. Este processo é essencial para garantir que a estrutura da base de dados está correta e que as relações entre as tabelas funcionam conforme o esperado. O povoamento inicial também permite testar consultas e operações de manipulação de dados, assegurando que a base de dados está pronta para ser utilizada num ambiente de produção.

Um dos aspetos mais críticos no povoamento de uma base de dados é a ordem pela qual as tabelas são povoadas. Esta ordem deve seguir uma sequência lógica que respeite as dependências entre as tabelas, de modo a evitar violações de integridade referencial. Por exemplo, não é possível inserir registos na tabela "Aluguer"antes de inserir registos nas tabelas "Veiculo"e "Cliente", pois a tabela "Aluguer"depende destas através de chaves estrangeiras. Este problema é semelhante ao que ocorre durante a criação da base de dados, onde a ordem de criação das tabelas deve respeitar as suas dependências.

Para resolver este problema, a equipa de desenvolvimento decidiu seguir uma ordem topológica baseada nas dependências entre as tabelas. Esta abordagem garante que cada tabela é povoada apenas após as tabelas das quais depende já terem sido povoadas, evitando assim violações de integridade referencial.

A equipa de desenvolvimento optou por utilizar a Linguagem de Manipulação de Dados (LMD) de SQL, especificamente as instruções INSERT, para adicionar à base de dados um conjunto de registos representativos. Este método foi escolhido devido à sua simplicidade e eficácia na inserção de dados em massa.

O processo de povoamento começou com a limpeza das tabelas existentes, seguindo uma ordem que respeita as dependências entre elas. Esta limpeza é necessária para garantir que a base de dados está num estado consistente antes de iniciar o povoamento.

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE Pagamento;
TRUNCATE TABLE Aluguer;
TRUNCATE TABLE Telemovel;
TRUNCATE TABLE Cliente;
TRUNCATE TABLE Veiculo;
TRUNCATE TABLE PostoAluguer;
SET FOREIGN_KEY_CHECKS = 1;
```

Após a limpeza das tabelas, o povoamento foi realizado seguindo a ordem topológica mencionada anteriormente. Cada tabela foi povoada com um conjunto de registos representativos, utilizando a instrução INSERT.

Por exemplo, a tabela "PostoAluguer"foi povoada com os seguintes registos:

```
INSERT INTO PostoAluguer VALUES
(1, 'Posto Central', 'Rua da Liberdade, 101', 912345678,
↪  'central@greenmotion.pt', 'Ativo'),
(2, 'Posto Norte', 'Av. do Minho, 99', 913456789,
↪  'norte@greenmotion.pt', 'Ativo'),
(3, 'Posto Sul', 'Rua Algarve, 22', 914567890,
↪  'sul@greenmotion.pt', 'Ativo');
```

Este processo foi repetido para todas as tabelas, seguindo sempre a ordem topológica definida. Os valores inseridos incluem diferentes tipos de dados, como inteiros, strings, datas e valores decimais. Por exemplo, na tabela "Veiculo", foram inseridos registos com valores inteiros para o ID e o número de lugares, strings para a marca e modelo, e valores decimais para a tarifa diária.

```
INSERT INTO Veiculo VALUES
(1, 1, 'AA-00-01', 'SUV', 'Tesla', 'Model X', 'Elétrico', 5, 500,
↪  'Ativo', 89.99),
(2, 1, 'BB-11-22', 'Citadino', 'Renault', 'ZOE', 'Elétrico', 4,
↪  300, 'Ativo', 49.99),
[...]
(14, 1, 'NN-54-64', 'Desportivo', 'Audi', 'e-tron GT', 'Elétrico',
↪  4, 430, 'Ativo', 124.99),
(15, 3, 'OO-65-75', 'SUV', 'Volvo', 'XC40 Recharge', 'Elétrico', 5,
↪  400, 'Ativo', 88.99);
```

O povoamento inicial da base de dados GreenMotionDB foi realizado com sucesso, seguindo uma ordem topológica que respeita as dependências entre as tabelas. Este processo garantiu que a base de dados está num estado consistente e pronta para ser utilizada em um ambiente de produção. O uso da Linguagem de Manipulação de Dados (LMD) de SQL permitiu inserir um conjunto de registos representativos de forma eficiente e eficaz, assegurando que a base de dados está pronta para ser testada e validada.

5.4 Cálculo do espaço da base de dados

A estimativa dos requisitos de armazenamento da base de dados GreenMotionDB foi realizada através de uma análise detalhada dos tipos de dados utilizados, do volume esperado de registos e das características específicas do motor de armazenamento InnoDB. Esta análise considera tanto os aspetos teóricos quanto as limitações práticas da implementação, permitindo determinar com precisão as necessidades de hardware e planear o crescimento futuro do sistema.

5.4.1 Caracterização dos Tipos de Dados

O motor de armazenamento InnoDB foi selecionado para todas as tabelas da base de dados. A tabela seguinte apresenta o espaço ocupado pelos diferentes tipos de dados utilizados no esquema:

Tipo de Dados	Espaço (Bytes)	Observações
INT	4	Chaves primárias e estrangeiras e outros valor numéricos
DECIMAL(10,2)	5	Valores monetários
VARCHAR(45)	1 - 46	Campos de texto curto
VARCHAR(200)	1 - 201	Moradas
VARCHAR(4)	1 - 5	Categoria da carta de condução
DATE	3	Validade da carta
DATETIME	8	Timestamps de alugueres e pagamentos
ENUM	1	Estados e tipos

Tabela 5.2: Espaço Ocupado pelos Tipos de Dados

5.4.2 Análise Teórica do Tamanho por Tabela

Para estabelecer uma base de comparação, foi primeiro calculado o espaço teórico ocupado por cada tabela, considerando valores médios realistas para os campos de tamanho variável.

PostoAluguer

Atributo	Tipo de Dados	Tamanho Médio	Espaço (Bytes)
ID	INT	-	4
Nome	VARCHAR(45)	25 chars	26
Morada	VARCHAR(200)	70 chars	71
NumTelefone	INT	-	4
Email	VARCHAR(45)	30 chars	31
Estado	ENUM	-	1
Total por registo			137 bytes

Tabela 5.3: Análise de um registo de PostoAluguer

Registos Esperados: 3

Total Teórico: 0.41 bytes

Veiculo

Atributo	Tipo de Dados	Tamanho Médio	Espaço (Bytes)
ID	INT	-	4
PostoAluguer_ID	INT	-	4
Matricula	VARCHAR(45)	8 chars	13
Categoria	VARCHAR(45)	12 chars	13
Marca	VARCHAR(45)	12 chars	13
Modelo	VARCHAR(45)	12 chars	13
TipoMotor	VARCHAR(45)	12 chars	13
NumLugares	INT	-	4
Autonomia	INT	-	4
Estado	ENUM	-	1
TarifaDiaria	DECIMAL(10,2)	-	5
Total por registo			83 bytes

Tabela 5.4: Análise de um registo de Veiculo

Registos Esperados: 120

Total Teórico: 9.96 kB

Cliente

Atributo	Tipo de Dados	Tamanho Médio	Espaço (Bytes)
ID	INT	-	4
Nome	VARCHAR(25)	30 chars	31
Tipo	ENUM	-	1
NIF	INT	-	4
Morada	VARCHAR(200)	65 chars	66
Email	VARCHAR(45)	28 chars	29
NumCartaCond	INT	-	4
CategoriaCartaCond	VARCHAR(4)	2 chars	3
ValidadeCartaCond	DATE	-	3
Estado	ENUM	-	1
Total por registo			146 bytes

Tabela 5.5: Análise de um registo de Cliente

Registos Esperados: 1000

Total Teórico: 146.0 kB

Telemovel

Atributo	Tipo de Dados	Tamanho Médio	Espaço (Bytes)
Numero	INT	-	4
Cliente_ID	INT	-	4
Total por registo			8 bytes

Tabela 5.6: Análise de um registo de Telemovel

Registos Esperados: 1200

Total Teórico: 9.6 kB

Aluguer

Atributo	Tipo de Dados	Tamanho Médio	Espaço (Bytes)
ID	INT	-	4
Veiculo_ID	INT	-	4
Cliente_ID	INT	-	4
DataHoraInicio	DATETIME	-	8
DataHoraFim	DATETIME	-	8
Estado	ENUM	-	1
Total por registo			29 bytes

Tabela 5.7: Análise de um registo de Aluguer

Registos Esperados: 8000

Total Teórico: 232.0 kB

Pagamento

Atributo	Tipo de Dados	Tamanho Médio	Espaço (Bytes)
ID	INT	-	4
Cliente_ID	INT	-	4
Aluguer_ID	INT	-	4
Montante	DECIMAL(10,2)	-	5
Metodo	VARCHAR(45)	15 chars	16
DataHora	DATETIME	-	8
Estado	ENUM	-	1
Total por registo			42 bytes

Tabela 5.8: Análise de um registo de Pagamento

Registos Esperados: 9000

Total Teórico: 378.0 kB

Resumo da Análise Teórica

Tabela	Registos	Tamanho Teórico
PostoAluguer	3	0.41 kB
Veiculo	120	9.96 kB
Cliente	1000	146.0 kB
Telemovel	1200	9.6 kB
Aluguer	8000	232.0 kB
Pagamento	9000	378.0 kB
Total	19.323	775.97 kB

Tabela 5.9: Análise de um registo de Pagamento

5.4.3 Análise Realista com a Arquitetura InnoDB

Conceito das Páginas de Dados

O InnoDB organiza todos os dados em páginas de 16 KB (16.384 bytes) como unidade fundamental de armazenamento. Cada tabela ocupa pelo menos uma página completa, independentemente do tamanho real dos dados armazenados. No entanto, cada página de 16 KB não está inteiramente disponível para dados do utilizador.

Componente	Espaço Ocupado
Cabeçalho	94 bytes
Trailer	8 bytes
Diretório	2 bytes por registo
Espaço reservado	5-10%
Espaço Útil Aproximado	15.700 bytes

Tabela 5.10: Análise de um registo de Pagamento

Cálculo Real por Tabela

Tendo o conceito de páginas de dados em consideração, é possível chegar a uma estimativa mais próxima da realidade.

Componente	Espaço Ocupado Teórico	Páginas Necessárias	Espaço Real
PostoAluguer	0.41 kB	1	16 kB
Veiculo	9.96 kB	1	16 kB
Cliente	146.0 kB	10	160 kB
Telemovel	9.6 kB	1	16 kB
Aluguer	232.0 kB	15	240 kB
Pagamento	378.0 kB	25	400 kB
Total	775.97 kB	52	848 kB

Tabela 5.11: Análise de um registo de Pagamento

A partir deste dados, estima-se um overhead de fragmentação próximo dos 10% (848 kB real vs 776 kB teórico).

5.4.4 Análise Completa dos Índices

A análise dos índices é fundamental para compreender o impacto no espaço ocupado e na performance do sistema. O esquema da base de dados GreenMotionDB contempla três grandes grupos de índices: chaves primárias, chaves estrangeiras e índices únicos/de performance.

As chaves primárias são essenciais para garantir a unicidade dos registos e a eficiência das operações CRUD. Cada tabela principal possui a sua própria chave primária, totalizando seis índices deste tipo. As chaves estrangeiras, também em número de seis, asseguram a integridade referencial entre tabelas relacionadas, como a ligação entre Veiculo e PostoAluguer, ou entre Pagamento e Aluguer.

Além destes, existem quatro índices únicos, criados para garantir a unicidade de campos críticos (como Nome do PostoAluguer, NIF e NumCartaCond do Cliente, e Matricula do Veiculo), bem como para otimizar consultas frequentes nesses campos.

O espaço ocupado pelos índices foi estimado tendo em conta o número médio de registos e a estrutura das páginas InnoDB.

Tipo de índices	Quantidade	Páginas Estimadas	Espaço Ocupado
Chaves Primárias	6	18	288 kB
Chaves Estrangeiras	6	18	288 kB
Índices Únicos	4	4	64 kB
Total	16	42	848 kB

Tabela 5.12: Análise do espaço dos Índices

5.4.5 Dimensão Total e Overhead do Sistema

O cálculo da dimensão total do sistema considera não apenas os dados das tabelas e os índices, mas também o overhead introduzido pelo InnoDB. Os dados das tabelas ocupam 848 kB, enquanto os índices consomem 448 kB, perfazendo um total de 1.296 kB para a base de dados operacional.

No entanto, o InnoDB acrescenta ainda componentes de overhead, como metadados do tablespace (128 kB), undo logs (265 kB), redo logs (512 kB) e estruturas de controlo do buffer pool (64 kB). Estes componentes, mesmo em volumes moderados, podem representar cerca de 960 kB adicionais. Assim, a estimativa final para o espaço ocupado pelo sistema atinge 2.256 kB (ou aproximadamente 2,2 MB), já incluindo todos os fatores de overhead relevantes.

Componentes	Espaço Relativo (%)	Espaço Absoluto
Dados	38%	848 kB
Índices	20%	448 kB
Overhead do Sistema	42%	960 kB
Total		2.256 kB (2.2 MB)

Tabela 5.13: Análise da Dimensao Total

5.4.6 Projeções de Crescimento

Para avaliar a evolução da base de dados GreenMotionDB ao longo de cinco anos, considerou-se uma evolução anual de 10% para clientes e telemóveis, 30% para alugueres e pagamentos, e 5% para veículos. Estes rácios traduzem um crescimento mais acelerado na utilização do serviço (alugueres e pagamentos), acompanhado de um aumento moderado na base de clientes e frota. Optou-se por manter o número de postos de aluguer estável uma vez que dentro de um cenário realista mesmo que novos postos sejam registados, isto não resultaria num aumento do número de páginas usadas para a tabela.

Evolução dos Registos ao Longo de 5 Anos

A tabela seguinte apresenta a evolução estimada dos registos (exceto PostoAluguer) ao longo de cinco anos:

Estimativa do Espaço Ocupado pelos Dados (Ano 5)

O espaço real ocupado será sempre arredondado para o múltiplo de página InnoDB (16 kB) imediatamente superior.

Ano	Clientes	Telemóveis	Alugueres	Pagamentos	Veículos
0	1000	1200	8000	9000	120
1	1100	1320	10400	11700	126
2	1210	1452	13520	15210	132
3	1331	1597	17576	19773	139
4	1464	1757	22849	25705	146
5	1611	1933	29704	33417	153

Tabela 5.14: Caption

Tabela	Registos	Espaço Real
PostoAluguer	3	16 kB
Veiculo	153	16 kB
Cliente	1611	240 kB
Telemovel	1933	16 kB
Aluguer	29704	864 kB
Pagamento	33417	1408 kB
Total		2560 kB (2.5 MB)

Tabela 5.15: Espaço ocupado pelos dados após 5 anos

Crescimento dos Índices

Os índices acompanham o crescimento dos dados. Mantendo a proporção observada (cerca de 35% do espaço dos dados), ao fim de cinco anos os índices ocuparão aproximadamente 900 kB.

Overhead do Sistema

O overhead do InnoDB mantém-se estável em cerca de 960 kB, pois não cresce linearmente com o número de registos.

Estimativa do Espaço Ocupado pelo Sistema (Ano 5)

Componentes	Espaço Relativo (%)	Espaço Absoluto
Dados	58%	2560 kB
Índices	20%	900 kB
Overhead do Sistema	22%	960 kB
Total		4420 kB (4.4 MB)

Tabela 5.16: Análise do espaço dos Índices

5.4.7 Considerações e Recomendações

A análise efetuada ao espaço necessário para a base de dados GreenMotionDB revela uma implementação eficiente, com um volume inicial modesto (cerca de 2,2 MB) e um crescimento controlado ao longo dos primeiros cinco anos de operação, atingindo aproximadamente 4,4 MB. Esta dimensão é perfeitamente comportável em qualquer sistema moderno, mesmo com recursos limitados.

A arquitetura baseada em InnoDB mostra-se validada: continua extremamente leve (menos de 6 MB após cinco anos), melhora em eficiência com volumes maiores e mantém excelente capacidade de escalar.

5.5 Definição e caracterização de vistas de utilização em SQL

O uso de vistas (views) em SQL permite criar sub esquemas lógicos a partir das tabelas reais da base de dados, servindo tanto como mecanismo de abstração como de controlo de acessos. Uma vista pode ser entendida como uma tabela virtual temporária, cujo conteúdo resulta de uma ou mais operações de seleção, projeção, junção ou agregação sobre as tabelas físicas. Em muitos sistemas de bases de dados, as vistas são utilizadas como uma forma de limitar o acesso dos utilizadores a determinados atributos ou registos, facilitando a implementação de políticas de segurança.

No âmbito da base de dados desenvolvida para a empresa GreenMotion, foi criada a vista VeiculosComDetalhes, de modo a facilitar o acesso consolidado a informação dos veículos, associando os dados técnicos de cada veículo ao nome do posto de aluguer onde se encontra. Esta vista pode ser útil em diversos contextos da aplicação, nomeadamente para visualização rápida de veículos disponíveis, interface de gestão ou mesmo para sistemas de consulta com permissões limitadas (como, por exemplo, operadores que apenas visualizam dados de frota). A sua criação em SQL foi efetuada da seguinte forma:

```
CREATE OR REPLACE VIEW VeiculosComDetalhes AS
SELECT v.ID, v.Matricula, v.Marca, v.Modelo, v.Categoria, p.Nome AS
↪  Posto
FROM Veiculo v
JOIN PostoAluguer p ON v.PostoAluguer_ID = p.ID
WHERE v.Estado = 'Ativo';
```

A vista realiza uma junção entre a tabela Veiculo e PostoAluguer, selecionando apenas veículos cujo estado seja 'Ativo'. Com esta projeção, os utilizadores podem visualizar diretamente o identificador do veículo, matrícula, marca, modelo, categoria e nome do posto em que se encontra estacionado. Esta vista foi utilizada numa das interrogações incluídas no ficheiro queries_calls.sql, nomeadamente:

```
SELECT * FROM VeiculosComDetalhes WHERE Categoria = 'SUV';
```

Este exemplo mostra a reutilização da vista para obter todos os veículos da categoria SUV com os detalhes do seu posto de origem. Esta abordagem simplifica a consulta e evita repetição de lógica SQL nas aplicações que consomem os dados da base de dados.

A equipa de desenvolvimento optou por não criar múltiplas vistas específicas por tipo de utilizador (como gestor, colaborador e admin), uma vez que o controlo de permissões foi já bem definido acima e não foi encontrada uma necessidade da implementação das vistas com tal finalidade. No entanto, a criação da vista VeiculosComDetalhes serve como exemplo da versatilidade e utilidade das vistas, podendo ser estendida futuramente para incluir filtros adicionais (ex: autonomia mínima, tipo de motor) ou ser utilizada em combinação com procedures e funções para relatórios.

Desta forma, cumpre-se o objetivo de exemplificar o uso de vistas de utilização em SQL, com um caso prático relevante para a aplicação e com ligação concreta a uma das interrogações do sistema.

5.6 Tradução das interrogações do utilizador para SQL

Estando já implementadas em álgebra relacional as interrogações à base de dados pedidas pelos requisitos de manipulação, a sua conversão para SQL é, na maior parte dos casos, uma tradução direta da sua estrutura lógica para a respectiva sintaxe. Nesta seção, serão apresentadas as instruções SQL na mesma ordem dos requisitos a que correspondem, sem justificar as operações utilizadas, pois estas encontram-se explicadas na seção de validação do modelo com interrogações.

RM-009 refere-se à obtenção dos veículos disponíveis num intervalo de tempo. A sua implementação requer uma seleção sobre a tabela Veiculo com exclusão de todos os veículos que já tenham um aluguer iniciado ou agendado nesse intervalo. Esta exclusão foi feita através de uma subconsulta NOT IN. A procedure VeiculosDisponiveis recebe dois parâmetros correspondentes às datas de início e fim do intervalo:

```
DELIMITER $$
CREATE PROCEDURE VeiculosDisponiveis(IN DataInicio DATETIME, IN
↪  DataFim DATETIME)
BEGIN
    SELECT * FROM Veiculo v
    WHERE v.Estado = 'Ativo' AND v.ID NOT IN (
        SELECT a.Veiculo_ID FROM Aluguer a
        WHERE a.Estado IN ('Iniciado', 'Agendado')
        AND (a.DataHoraInicio <= DataFim AND a.DataHoraFim >=
        ↪  DataInicio)
```

```
    );
END $$
DELIMITER ;
CALL VeiculosDisponiveis('2025-06-01 10:00:00', '2025-06-05
↪  10:00:00');
```

RM-010, que corresponde à enumeração de veículos por características técnicas, requer a aplicação de várias seleções condicionais em atributos da tabela Veiculo. Foi implementada através de uma procedure que aceita como parâmetros os filtros desejados, e interpreta o valor NULL como "ignorar este critério". Esta solução permite ao utilizador consultar por qualquer combinação de atributos, como mostra o exemplo abaixo:

```
DELIMITER $$
CREATE PROCEDURE VeiculosPorCaracteristicas(
    IN pCategoria VARCHAR(45), IN pMarca VARCHAR(45), IN pModelo
     ↪  VARCHAR(45),
    IN pTipoMotor VARCHAR(45), IN pNumLugares INT
)
BEGIN
    SELECT * FROM Veiculo
    WHERE Estado = 'Ativo'
    AND (Categoria = pCategoria OR pCategoria IS NULL)
    AND (Marca = pMarca OR pMarca IS NULL)
    AND (Modelo = pModelo OR pModelo IS NULL)
    AND (TipoMotor = pTipoMotor OR pTipoMotor IS NULL)
    AND (NumLugares = pNumLugares OR pNumLugares IS NULL);
END $$
DELIMITER ;
CALL VeiculosPorCaracteristicas('SUV', 'Tesla', NULL, 'Elétrico',
↪  NULL);
```

RM-011, relativo ao histórico de aluguer de um determinado cliente, consiste numa junção entre Aluguer e Veiculo, filtrando os resultados pelo Cliente_ID. Os registos são apresentados por ordem decrescente da data de início:

```
DELIMITER $$
CREATE PROCEDURE HistoricoAluguerCliente(IN ClienteID INT)
BEGIN
    SELECT a.*, v.Marca, v.Modelo
    FROM Aluguer a JOIN Veiculo v ON a.Veiculo_ID = v.ID
    WHERE a.Cliente_ID = ClienteID
    ORDER BY a.DataHoraInicio DESC;
END $$
DELIMITER ;
CALL HistoricoAluguerCliente(1);
```

RM-012 trata da identificação de um cliente a partir do seu número de telemóvel. Para isso, é efetuada uma junção entre a tabela Cliente e a tabela Telemovel, filtrando pela coluna

Numero. Esta interrogação é útil para sistemas de apoio ao cliente, em que o número de telefone é frequentemente o único dado disponível para pesquisa:

```
DELIMITER $$
CREATE PROCEDURE ClientePorTelemovel(IN NumeroTelemovel INT)
BEGIN
    SELECT c.* FROM Cliente c JOIN Telemovel t ON c.ID =
    ↪  t.Cliente_ID
    WHERE t.Numero = NumeroTelemovel;
END $$
DELIMITER ;
CALL ClientePorTelemovel(912345678);
```

RM-013, referente à receita gerada por um determinado posto de aluguer entre duas datas, requer a utilização de múltiplas junções entre as tabelas Pagamento, Aluguer, Veiculo e PostoAluguer. A soma dos montantes dos pagamentos realizados no intervalo é feita com recurso à função SUM, agrupando os resultados por posto:

```
DELIMITER $$
CREATE PROCEDURE ReceitaPosto(IN PostoID INT, IN DataInicio
↪  DATETIME, IN DataFim DATETIME)
BEGIN
    SELECT pa.ID AS PostoID, pa.Nome, SUM(p.Montante) AS
    ↪  ReceitaTotal
    FROM Pagamento p
    JOIN Aluguer a ON p.Aluguer_ID = a.ID
    JOIN Veiculo v ON a.Veiculo_ID = v.ID
    JOIN PostoAluguer pa ON v.PostoAluguer_ID = pa.ID
    WHERE p.Estado = 'Realizado'
    AND p.DataHora BETWEEN DataInicio AND DataFim
    AND pa.ID = PostoID
    GROUP BY pa.ID, pa.Nome;
END $$
DELIMITER ;
CALL ReceitaPosto(1, '2025-01-01', '2025-05-31');
```

RM-014 exige o cálculo da taxa de ocupação dos veículos de uma determinada categoria numa data fornecida. Esta interrogação foi implementada com recurso a uma função SQL (FUNCTION) que executa duas contagens: o número total de veículos ativos da categoria e o número de veículos ocupados nessa data. A função IF garante que, no caso de não existirem veículos da categoria, a função retorna 0, evitando o erro divisão por zero. A instrução SQL correspondente é:

```
DELIMITER $$
CREATE FUNCTION TaxaOcupacao(CategoriaVeiculo VARCHAR(45), DataRef
↪  DATETIME)
RETURNS DECIMAL(5,2)
```

```
DETERMINISTIC
BEGIN
    DECLARE Total INT DEFAULT 0;
    DECLARE Ocupados INT DEFAULT 0;
    DECLARE Taxa DECIMAL(5,2);

    -- Total de veículos ativos da categoria
    SELECT COUNT(*) INTO Total
    FROM Veiculo
    WHERE Categoria = CategoriaVeiculo AND Estado = 'Ativo';

    -- Veículos ocupados naquele instante, considerando qualquer
    ↪  aluguer NÃO cancelado
    SELECT COUNT(DISTINCT v.ID) INTO Ocupados
    FROM Veiculo v
    JOIN Aluguer a ON v.ID = a.Veiculo_ID
    WHERE v.Categoria = CategoriaVeiculo
      AND v.Estado = 'Ativo'
      AND a.DataHoraInicio <= DataRef
      AND a.DataHoraFim >= DataRef
      AND a.Estado <> 'Cancelado';

    SET Taxa = IF(Total = 0, 0, (Ocupados * 100.0) / Total);
    RETURN Taxa;
END$$
DELIMITER ;
SELECT TaxaOcupacao('SUV', '2025-06-01 12:00:00') AS
↪  TaxaOcupacaoPercentual;
```

Todas estas interrogações foram também incluídas no ficheiro queries_calls.sql do projeto, com chamadas parametrizadas e comentários explicativos. A sua execução foi testada com dados simulados na base de dados, confirmando-se que os resultados produzidos estavam em conformidade com os requisitos definidos.

5.7 Indexação do Sistema de Dados

5.7.1 Visão Geral do Sistema de Indexação

Os índices são estruturas de dados que melhoram significativamente o desempenho das consultas numa base de dados, permitindo localizar registos sem necessidade de percorrer toda a tabela. Embora introduzam overhead nas operações de inserção e atualização, e ocupem espaço adicional de armazenamento, os benefícios em termos de velocidade de consulta justificam geralmente a sua utilização.

Na base de dados GreenMotion Solutions, devido ao volume reduzido de dados na fase inicial, não foram implementados índices adicionais para além das chaves primárias e restrições UNIQUE automaticamente indexadas pelo sistema. No entanto, foi preparado o código para criação de índices que poderá ser executado futuramente caso o crescimento da base de dados o justifique.

5.7.2 Índices Propostos

Analisando as consultas mais prováveis no sistema, identificaram-se os seguintes índices benéficos:

Para consultas de veículos por posto de aluguer:

```
CREATE INDEX idx_veiculo_posto ON Veiculo(PostoAluguer_ID);
```

Para pesquisas de alugueres por cliente:

```
CREATE INDEX idx_aluguer_cliente ON Aluguer(Cliente_ID);
```

Para consultas de alugueres por veículo:

```
CREATE INDEX idx_aluguer_veiculo ON Aluguer(Veiculo_ID);
```

Para análises temporais de alugueres:

```
CREATE INDEX idx_aluguer_data_inicio ON Aluguer(DataHoraInicio);
```

Para consultas de pagamentos por cliente:

```
CREATE INDEX idx_pagamento_cliente ON Pagamento(Cliente_ID);
```

Para associação de pagamentos a alugueres:

```
CREATE INDEX idx_pagamento_aluguer ON Pagamento(Aluguer_ID);
```

Para contactos de clientes:

```
CREATE INDEX idx_telemovel_cliente ON Telemovel(Cliente_ID);
```

Observa-se que a maioria dos índices propostos corresponde a chaves estrangeiras, seguindo a prática comum de indexar estas colunas para otimizar operações de junção entre tabelas.

5.7.3 Avaliação de Desempenho

Com o volume atual de dados, não foram detetadas diferenças percetíveis de desempenho. A equipa considera que seria necessário um volume de dados significativamente maior para

justificar a implementação imediata destes índices.

Numa fase de produção com maior volume de dados, cada índice deverá ser avaliado individualmente quanto ao seu impacto real no desempenho, considerando tanto os benefícios nas consultas como os custos nas operações de escrita e manutenção.

5.8 Implementação de Procedimentos, Funções e Gatilhos

MySQL suporta a definição e execução de procedimentos, funções e gatilhos, que permitem, dentro do próprio SGBD, o controlo de fluxo de execução de várias interrogações, sem a necessidade de software aplicacional para gerir operações condicionais e cíclicas. Estas estruturas permitem reforçar a consistência dos dados, encapsular lógica e evitar repetição de código.

5.8.1 Transações

Uma transação é um conjunto de operações que deve ser executado como uma unidade atómica: ou todas são executadas com sucesso, ou nenhuma é aplicada. Isto é particularmente importante quando várias alterações a diferentes tabelas devem ser realizadas em conjunto, como no caso de um aluguer que implica a criação de um registo na tabela `Aluguer` e um pagamento correspondente na tabela `Pagamento`.

Transações são iniciadas com a instrução `STARTTRANSACTION` e finalizadas com `COMMIT` (se bem-sucedidas) ou `ROLLBACK` (em caso de falha).

Abaixo apresenta-se um exemplo simples de uma transação utilizada para garantir consistência na remoção de um veículo e dos seus alugueres associados:

```
START TRANSACTION;

DELETE FROM Aluguer
WHERE Veiculo_ID = 7;

DELETE FROM Veiculo
WHERE ID = 7;

COMMIT;
```

Este bloco garante que, caso ocorra um erro durante uma das remoções, nenhuma delas será aplicada, evitando a existência de alugueres órfãos ou inconsistências no sistema.

5.8.2 Procedimentos

Procedimentos armazenados (ou *stored procedures*) são blocos de código reutilizáveis que encapsulam operações sobre a base de dados. São úteis para uniformizar operações, reduzir código duplicado na aplicação e, quando necessário, conter transações internas.

Como exemplo, apresenta-se a procedure `RemoverVeiculo`, que elimina um veículo da base de dados após apagar os alugueres associados:

```
DELIMITER $$
CREATE PROCEDURE RemoverVeiculo(IN pID INT)
BEGIN
    START TRANSACTION;

    DELETE FROM Aluguer WHERE Veiculo_ID = pID;
    DELETE FROM Veiculo WHERE ID = pID;

    COMMIT;
END $$
DELIMITER ;
```

Neste caso, a operação está protegida por uma transação, garantindo que o veículo só é removido se também for possível remover todos os alugueres a ele associados. Caso contrário, o `ROLLBACK` poderia ser invocado para anular a operação (não incluído neste exemplo por simplicidade).

Foram ainda criados outros procedimentos como `InserirCliente`, `AtualizarCliente`, `ReceitaPosto`, e `VeiculosDisponiveis`, todos com o objetivo de reutilização da lógica de negócio e separação de responsabilidades.

5.8.3 Funções

Além dos procedimentos, foi também implementada uma função, `TaxaOcupacao`, com o objetivo de calcular a percentagem de ocupação de veículos de uma determinada categoria numa data específica, conforme requerido no RM-014. Esta função recebe dois argumentos e retorna um valor decimal com a percentagem de ocupação.

```
DELIMITER $$

CREATE FUNCTION TaxaOcupacao(CategoriaVeiculo VARCHAR(45), DataRef
↪   DATETIME)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE Total INT DEFAULT 0;
```

```
    DECLARE Ocupados INT DEFAULT 0;
    DECLARE Taxa DECIMAL(5,2);

    -- Total de veículos ativos da categoria
    SELECT COUNT(*) INTO Total
    FROM Veiculo
    WHERE Categoria = CategoriaVeiculo AND Estado = 'Ativo';

    -- Veículos ocupados naquele instante, considerando qualquer
     ↪  aluguer NÃO cancelado
    SELECT COUNT(DISTINCT v.ID) INTO Ocupados
    FROM Veiculo v
    JOIN Aluguer a ON v.ID = a.Veiculo_ID
    WHERE v.Categoria = CategoriaVeiculo
      AND v.Estado = 'Ativo'
      AND a.DataHoraInicio <= DataRef
      AND a.DataHoraFim >= DataRef
      AND a.Estado <> 'Cancelado';

    SET Taxa = IF(Total = 0, 0, (Ocupados * 100.0) / Total);
    RETURN Taxa;
END$$

DELIMITER ;
```

Esta função permite consultar rapidamente o grau de utilização da frota de veículos e pode ser incorporada em relatórios e dashboards operacionais. A escolha por uma função justifica-se pela simplicidade da operação, necessidade de retorno direto de valor, e possibilidade de reutilização em interrogações SQL.

5.8.4 Gatilhos

Para garantir a integridade dos dados e o cumprimento de regras de negócio durante operações de inserção, foram definidos dois gatilhos do tipo `BEFOREINSERT`, incluídos no ficheiro `triggers.sql`.

O primeiro gatilho, `prevent_double_booking`, assegura que um veículo não possa ser reservado para um período em que já se encontre associado a outro aluguer ativo ou agendado. Para isso, verifica se o novo intervalo de datas fornecido colide com algum aluguer existente com estado `Agendado` ou `Iniciado`:

```
DELIMITER $$
CREATE TRIGGER prevent_double_booking
BEFORE INSERT ON Aluguer
FOR EACH ROW
BEGIN
```

```
  IF EXISTS (
    SELECT 1 FROM Aluguer
    WHERE Veiculo_ID = NEW.Veiculo_ID
      AND Estado IN ('Agendado', 'Iniciado')
      AND (
        (NEW.DataHoraInicio BETWEEN DataHoraInicio AND DataHoraFim)
        OR
        (NEW.DataHoraFim BETWEEN DataHoraInicio AND DataHoraFim)
        OR
        (DataHoraInicio BETWEEN NEW.DataHoraInicio AND
         ↪  NEW.DataHoraFim)
      )
  ) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Vehicle already booked for this period!';
  END IF;
END$$
DELIMITER ;
```

Esta verificação permite evitar sobreposições de agendamentos, protegendo tanto a integridade da base de dados como a experiência do utilizador final.

O segundo gatilho, `prevent_expired_license`, garante que um cliente com carta de condução expirada não possa realizar um novo aluguer. A validade da carta é consultada na tabela `Cliente`, e caso seja anterior à data atual, a operação é imediatamente anulada:

```
DELIMITER $$
CREATE TRIGGER prevent_expired_license
BEFORE INSERT ON Aluguer
FOR EACH ROW
BEGIN
  DECLARE validade DATE;
  SELECT ValidadeCartaCond INTO validade FROM Cliente WHERE ID =
   ↪  NEW.Cliente_ID;
  IF validade < CURDATE() THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot rent: Driver\'s license expired!';
  END IF;
END$$
DELIMITER ;
```

Ambos os gatilhos reforçam regras de integridade fundamentais e asseguram que essas regras são aplicadas de forma automática e consistente, independentemente da origem da operação de inserção (interface, script ou aplicação externa). A sua utilização reduz a necessidade de validação redundante no nível da aplicação e centraliza a lógica de negócio crítica no próprio SGBD.

5.8.5 Considerações Finais

A escolha da estrutura usada em cada requisito baseou-se no comportamento desejado e no tipo de operação envolvida:

- **Procedimentos** foram utilizados em situações que envolvem múltiplas instruções encadeadas ou que requerem lógica condicional, como inserção de alugueres e respetivos pagamentos;
- **Funções** foram empregues para operações de leitura com retorno único, reutilizáveis em relatórios ou análises estatísticas;
- **Gatilhos** foram definidos para reforçar regras de integridade nos momentos de inserção, assegurando consistência independentemente da origem dos dados.

A equipa de desenvolvimento optou por esta separação clara de responsabilidades para melhorar a organização do sistema, reduzir a duplicação de lógica e garantir maior fiabilidade na execução das operações sobre a base de dados.

6 Implementação do Sistema de Migração Dados

6.1 Visão Geral do Sistema de Migração

O sistema de migração de dados implementado tem como objetivo consolidar informações provenientes de três fontes distintas numa base de dados central MySQL denominada GreenMotionDB. Esta implementação permite a integração de dados heterogéneos, garantindo a integridade referencial e a consistência dos dados migrados através de um processo automatizado e robusto.

6.2 Caracterização das Fontes de Dados

6.2.1 Base de Dados PostgreSQL (clientesbd)

A primeira fonte de dados consiste numa base de dados relacional PostgreSQL que centraliza as informações sobre clientes e os respetivos contactos telefónicos. Esta base apresenta uma estrutura normalizada com duas tabelas principais: Cliente e Telemovel. A tabela de clientes armazena dados críticos incluindo informação fiscal (NIF), dados da carta de condução com categoria e validade, e estado operacional do cliente.

Os dados extraídos incluem a informação completa dos clientes com todos os campos relevantes para o sistema de aluguer de veículos, bem como os números de telemóvel associados através de uma relação one-to-many que permite múltiplos contactos por cliente. A extração é realizada através de queries SQL diretas que recuperam todos os registos ativos da base de dados origem.

6.2.2 Ficheiros CSV

A segunda fonte é constituída por três ficheiros CSV que representam diferentes entidades do sistema operacional: postos de aluguer, veículos e histórico de alugueres. Estes ficheiros mantêm uma estrutura tabular consistente com cabeçalhos definidos, permitindo processamento sistemático através da biblioteca pandas.

6.2.3 Ficheiro JSON

A terceira fonte consiste num ficheiro JSON (pagamentos.json) que contém as transações financeiras do sistema. Os dados de pagamento incluem referências aos clientes e alugueres correspondentes, valores monetários, métodos de pagamento, timestamps e estados de processamento, proporcionando um registo completo das transações financeiras do sistema.

6.3 Estratégia de Migração

6.3.1 Ordem de Migração

A migração implementa uma sequência cuidadosamente ordenada que respeita as dependências relacionais entre entidades. O processo inicia-se com as entidades independentes (PostoAluguer e Cliente), progride para as entidades dependentes de primeiro nível (Telemovel e Veiculo), continua com as entidades de segundo nível (Aluguer) que requerem referências múltiplas, e finaliza com as entidades de terceiro nível (Pagamento) que dependem de todo o contexto operacional.

Esta abordagem hierárquica é implementada através da função run_migration() que orquestra toda a sequência de forma estruturada, garantindo que cada etapa seja completada antes da progressão para a seguinte. O sistema mantém mapeamentos de identificadores ao longo de todo o processo para preservar as relações entre entidades.

6.3.2 Mapeamento de Chaves Primárias

Um dos principais desafios da migração reside no mapeamento entre as chaves primárias das fontes originais e as novas chaves auto-incrementais do MySQL. A solução implementada utiliza dicionários Python que mantêm a correspondência entre identificadores originais e destino para cada tabela com dependências. Este mecanismo garante que as relações entre entidades sejam preservadas durante todo o processo de migração, independentemente dos novos valores de chave primária gerados automaticamente pelo MySQL.

6.4 Tecnologias e Ferramentas Utilizadas

6.4.1 Linguagem de Programação: Python

A escolha do Python como linguagem principal justifica-se pelo seu excelente suporte para manipulação de dados através de bibliotecas especializadas, nomeadamente, psycopg2 para

conectividade PostgreSQL, mysql.connector para MySQL, pandas para processamento de CSV e json nativo para dados JSON e ainda pela facilidade de manutenção e legibilidade do código resultante.

6.4.2 Bibliotecas Especializadas

O sistema utiliza a biblioteca psycopg2 como conector para PostgreSQL, proporcionando conexão segura e eficiente à base de dados origem com suporte completo para transações e prepared statements. Para MySQL, emprega-se o mysql.connector, o conector oficial que oferece conectividade nativa e funcionalidades avançadas como LAST_INSERT_ID() essencial para o mapeamento de chaves.

A manipulação de dados tabulares é realizada através da biblioteca pandas, que oferece leitura e processamento eficiente de ficheiros CSV com estruturas de dados otimizadas para análise e transformação. O processamento de dados JSON utiliza a biblioteca nativa do Python, garantindo conversão direta entre objetos Python e JSON sem dependências externas. Para manipulação temporal, o sistema emprega a biblioteca datetime, assegurando conversão consistente entre diferentes formatos de data das fontes e garantindo consistência temporal nos dados migrados.

6.5 Implementação e Execução do Script

6.5.1 Funções Centrais de Processamento

A função insert_with_id_mapping() implementa a funcionalidade central do sistema, processando cada registo individualmente com captura automática de novos identificadores. A função utiliza transações individuais para cada inserção, permitindo rollback granular em caso de falhas sem afetar registos já processados com sucesso. O processo inclui validação automática através de blocos try-except que capturam erros de inserção, executam rollback da transação específica, e continuam o processamento com os registos restantes.

A função insert_simple_data() implementa processamento otimizado para entidades que não requerem mapeamento de identificadores, utilizando executemany() para inserções em lote quando possível. Esta abordagem melhora significativamente a performance para entidades como Telemovel e Pagamento que dependem apenas de referências já estabelecidas.

6.5.2 Sequência de Migração e Processamento de Entidades

A primeira fase processa o ficheiro postosdealuguer.csv através de `migrate_postos_from_csv()`, criando o mapeamento fundamental para referências de localização. A função utiliza

pandas para leitura estruturada do CSV, aplica conversões de tipo apropriadas (int para IDs e números de telefone), e normaliza estados através de `capitalize()`.

A segunda fase extrai dados de clientes diretamente do PostgreSQL através de `migrate_clientes_from_postgres()`, utilizando uma query SQL que recupera todos os campos relevantes. A função aplica transformações consistentes incluindo capitalização de tipos e estados, conversão para uppercase das categorias de carta de condução, e tratamento de valores nulos.

A terceira fase processa contactos telefónicos através de `migrate_phone_numbers_from_postgres()`, que utiliza o mapeamento de clientes estabelecido na fase anterior. A função implementa filtragem automática onde apenas números associados a clientes migrados com sucesso são processados, garantindo integridade referencial. A quarta fase processa dados de veículos através de `migrate_vehicles_from_csv()`, que referencia postos através do mapeamento criado na primeira fase. A função implementa validação prévia onde apenas veículos associados a postos válidos são processados, aplicando conversões de tipo e valores padrão apropriados.

A quinta fase migra o histórico de alugueres através de `migrate_rentals_from_csv()`, que implementa uma versão especializada do processamento com mapeamento. Esta função requer referências tanto a veículos como a clientes, demonstrando a importância da sequência hierárquica de migração. O processamento inclui conversão de timestamps para objetos datetime nativos.

A fase final processa transações financeiras através de `migrate_payments_from_json()`, que utiliza mapeamentos tanto de clientes como de alugueres. A função implementa parsing JSON com tratamento de diferentes formatos de timestamp através de `fromisoformat()`, e validação de referências antes da inserção.

6.5.3 Transformações de Dados e Tratamento de Erros

O sistema implementa transformações consistentes aplicadas durante o processamento, incluindo normalização de estados através de capitalize(), conversão de categorias de carta de condução para uppercase(), e tratamento cuidadoso de campos opcionais com valores None. Para dados temporais, o sistema utiliza datetime.strptime() e datetime.fromisoformat() para conversão consistente entre diferentes formatos presentes nas fontes.

O sistema implementa recuperação a múltiplos níveis começando com processamento individual de registos. A validação prévia de referências implementada em todas as funções de migração específicas verifica se os identificadores referenciados existem nos mapeamentos antes de tentar inserções dependentes. Esta abordagem evita violações de integridade referencial e proporciona diagnósticos claros sobre problemas de consistência.

7 Conclusões e Trabalho Futuro

O desenvolvimento do Sistema de Base de Dados para a GreenMotion Solutions foi concluído com sucesso, apesar de desvios em relação ao plano inicial delineado no diagrama de Gantt. Alterações, como a exclusão do dashboard em Power BI a pedido da empresa, foram integradas ao fluxo de trabalho sem comprometer a entrega do sistema dentro do tempo estabelecido. Esses ajustes, embora não críticos, demonstram a flexibilidade da equipa em adaptar-se a mudanças no escopo sem perder o foco nos objetivos principais.

Embora o sistema já esteja implementado, o impacto real nas operações da empresa ainda não foi totalmente avaliado. Espera-se que a solução venha a melhorar significativamente a gestão de dados, promovendo maior eficiência e precisão nos processos operacionais. No entanto, uma análise mais abrangente será necessária para compreender plenamente os benefícios e identificar possíveis áreas de melhoria.

Este trabalho não apenas atende às necessidades imediatas da GreenMotion Solutions, mas também estabelece uma base sólida para futuras expansões e melhorias. Entre as oportunidades para continuidade, destaca-se o potencial desenvolvimento do dashboard em Power BI, que permitirá análises avançadas e suporte ao planeamento estratégico. Assim, o projeto representa um marco inicial que reforça o compromisso da empresa com a modernização e a sustentabilidade, alinhando suas operações com as melhores práticas do setor.

PLANO DE EXECUÇÃO

			MARÇO				ABRIL					MAIO				JUNHO
Tarefas	Envolvidos	SEMANA / TEMPO	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Definição de Requisitos																CONCLUSÃO DO PROJETO
Definição do método	Analista	4 horas														
Levantamento de requisitos	Analista	22 horas														
Análise e organização dos requisitos	Analista e Arquiteto	16 horas														
Validação dos requisitos	Analista e Arquiteto	4 horas														
Modelação Concetual																
Identificação das entidades e relacionamentos	Arquiteto	6 horas														
Definição de atributos	Arquiteto	4 horas														
Elaboração do modelo concetual	Arquiteto	3 horas														
Validação do modelo concetual	Analista e Arquiteto	4 horas														
Modelação Lógica																
Derivação do modelo lógico	Arquiteto	4 horas														
Validação do modelo lógico	Arquiteto e Engenheiro	4 horas														
Implementação																
Criação do sistema de dados	Engenheiro	12 horas														
Implementação de procedimentos	Engenheiro	12 horas														
Integração dos dados	Engenheiro e Administrador	16 horas														
~~Desenvolvimento do dashboard~~	Engenheiro e Administrador	~~14 horas~~														
Validação da base de dados	Engenheiro e Administrador	8 horas														

Figura 7.1: Diagrama de Gantt do Fluxo de Trabalho

Referências

[1] Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed. – Global ed.). Pearson Education Limited.

[2] Cândido, C. (2020). brModelo. SIS4.com. http://www.sis4.com/brModelo/

[3] Oracle. (2024b). MySQL Workbench. MySQL. https://www.mysql.com/products/workbench/

[4] Il-Yeol, S., Evans, M., Park, E.K. (1995) A Comparative Analysis of Entity-Relationship Diagrams. Journal of Computer and Software Engineering, Vol. 3, No.4, pp. 427-459. https://cci.drexel.edu/faculty/song/publications/p_Jcse-erd.PDF

Lista de Siglas e Acrónimos

Lista com todas as siglas e acrónimos utilizados durante a realização do relatório:

BD Base de Dados

CEO Chief Executive Officer

CSV Comma-Separated Values

ER Entidade-Relacionamento

FN Forma Normal

ID Identificação

JSON JavaScript Object Notation

NIF Número de Identificação Fiscal

RC Requisitos de Controlo

RD Requisitos de Descrição

RM Requisitos de Manipulação

SBD Sistema de Base de Dados

SGBD Sistema de Gestão de Base de Dados

TI Tecnologias Da Informação

Anexos

Anexo 1 - Tabela de Requisitos Levantados

REQUISITOS LEVANTADOS						
Identificador	Data	Hora	Descrição	Fonte	Contexto	Responsável
R-001	14/03/2025	10:00	Precisamos de um sistema para gerir o aluguer de veículos com diferentes características técnicas.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-002	14/03/2025	10:03	Cada veículo deve ter um código único, matrícula, categoria, marca e outras características importantes como tipo de motor e número de lugares.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-003	14/03/2025	10:12	Os clientes podem ser particulares ou empresas, e precisamos de registar todos os seus dados pessoais.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-004	14/03/2025	10:15	É obrigatório registar o NIF e os dados da carta de condução de cada cliente, incluindo a validade.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-005	14/03/2025	10:18	Precisamos de pelo menos um contacto telefónico de cada cliente e, se possível, um email.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-006	14/03/2025	10:36	Precisamos de registar quando um aluguer começa e quando termina.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-007	14/03/2025	10:42	Os pagamentos devem ser registados com valores, método de pagamento e data.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-008	14/03/2025	10:54	Cada veículo pertence a um único posto de aluguer, mas pode ser alugado várias vezes.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-009	14/03/2025	10:57	Um cliente pode alugar vários veículos e fazer vários pagamentos ao longo do tempo.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-010	14/03/2025	11:00	Quando um cliente aluga um veículo, o sistema deve criar automaticamente um pagamento.	Daniela Rodrigues	Entrevista	Marco Sèvegrand

R-011	14/03/2025	11:03	O sistema deve calcular automaticamente quanto o cliente deve pagar baseado no período de aluguer e na tarifa diária.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-012	14/03/2025	11:06	Precisamos de poder procurar veículos disponíveis para um determinado período.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-013	14/03/2025	11:09	Os colaboradores devem conseguir encontrar veículos com características técnicas específicas.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-014	14/03/2025	11:12	Devemos poder consultar o histórico de alugueres de um cliente.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-015	14/03/2025	12:14	Precisamos de encontrar rapidamente um cliente pelo seu número de telemóvel.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-016	14/03/2025	11:03	A gestão necessita de saber a receita gerada por cada posto de aluguer em cada mês.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-017	14/03/2025	11:06	Precisamos de saber a taxa de ocupação de cada categoria de veículo.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-018	14/03/2025	11:09	Um veículo não pode estar alugado a mais de um cliente ao mesmo tempo.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-019	14/03/2025	11:12	Os clientes particulares só podem ter dois alugueres ativos em simultâneo.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-020	14/03/2025	12:14	Precisamos de diferentes níveis de acesso para diferentes funções na empresa.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-021	14/03/2025	11:03	Só o coordenador de gestão de frota pode adicionar, modificar ou remover veículos do sistema.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-022	14/03/2025	11:06	O diretor de operações é o único que deve gerir os postos de aluguer.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-023	14/03/2025	11:09	Apenas o diretor de vendas pode eliminar registos de clientes, alugueres e pagamentos.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-024	14/03/2025	11:12	Qualquer colaborador deve poder registar novos clientes e atualizar os dados deles.	Cristina Ferreida	Entrevista	Marco Sèvegrand

R-025	14/03/2025	12:14	Todos os colaboradores devem poder registar alugueres e pagamentos, mas só podem mudar o estado destes.	Daniela Rodrigues	Entrevista	Marco Sèvegrand
R-026	14/03/2025	11:03	Quando um veículo é devolvido, o sistema deve registar o fim do aluguer.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-027	14/03/2025	11:06	Queremos poder cancelar alugueres e processar reembolsos quando necessário.	Cristina Ferreida	Entrevista	Marco Sèvegrand
R-028	14/03/2025	16:03	Devemos saber a autonomia de cada veículo e quanto custa alugá-lo por dia.	Sistemas de Informação	Análise	Marco Sèvegrand
R-029	14/03/2025	16:12	Precisamos de diferenciar veículos ativos e inativos no sistema.	Sistemas de Informação	Análise	Marco Sèvegrand
R-030	14/03/2025	16:23	A empresa tem vários postos de aluguer espalhados que precisam ser geridos no sistema.	Sistemas de Informação	Análise	Marco Sèvegrand
R-031	14/03/2025	16:26	Cada posto deve ter um código de identificação, nome, morada completa e contactos.	Sistemas de Informação	Análise	Marco Sèvegrand
R-032	14/03/2025	16:35	Um posto pode estar ativo ou inativo no sistema.	Sistemas de Informação	Análise	Marco Sèvegrand
R-033	14/03/2025	16:39	Um aluguer pode ter diferentes estados: agendado, iniciado, terminado ou cancelado.	Sistemas de Informação	Análise	Marco Sèvegrand
R-034	14/03/2025	16:46	Um pagamento pode estar pendente, realizado, cancelado ou até mesmo devolvido.	Sistemas de Informação	Análise	Marco Sèvegrand

Anexo 2 - Tabela de Requisitos de Descrição

REQUISITOS DE DESCRIÇÃO		
Req.Descrição		
Identificador	Descrição	Revisor
RD-001	Um veículo é caracterizado por um código de identificação único (campo ID, gerado automaticamente), matrícula (Matricula), categoria (Categoria), marca (Marca), modelo (Modelo), tipo de motor (TipoMotor, como elétrico, híbrido...), número de lugares (NumLugares), autonomia (Autonomia), estado (Estado: Ativo ou Inativo) e tarifa diária (TarifaDiaria).	Marco Sèvegrand
RD-002	Um cliente é caracterizado por um código de idenficação único (numérico sequencial, exemplo: '00001'), o seu tipo (particular ou empresa), nome completo, número de de idenficação fiscal (NIF), carta de condução (número, validade e categoria), morada completa, pelo menos um número de telemóvel, email opcional e estado (ativo ou inativo).	Marco Sèvegrand
RD-003	Um posto de aluguer é caracterizado por um código de idenficação único (numérico sequencial, exemplo: '00001'), nome, morada completa, número de telefone, email e estado (ativo ou inativo).	Marco Sèvegrand
RD-004	Um aluguer é caracterizado por um um código de idenficação único (numérico sequencial, exemplo: '00001'), data de início e fim e estado (agendado, iniciado, terminado ou cancelado).	Marco Sèvegrand
RD-005	Um pagamento é caracterizado por um código de identificação único (numérico sequencial, exemplo: '00001'), montante pago, método de pagamento, data de realização (que é nula enquanto o pagamento não for efetuado) e estado (pendente, realizado, cancelado, devolvido).	Marco Sèvegrand
RD-006	Um veículo é gerido por um único posto de aluguer e pode participar em um ou mais alugueres.	Marco Sèvegrand
RD-007	Um posto de aluguer pode gerir um ou mais veículos.	Marco Sèvegrand
RD-008	Um cliente pode solicitar um ou mais alugueres e efetuar um ou mais pagamentos.	Marco Sèvegrand
RD-009	Um aluguer tem associado um veículo e cliente e pode envolver um ou mais pagamentos.	Marco Sèvegrand
RD-010	Um pagamento tem associado um aluguer e é efetuado por um cliente.	Marco Sèvegrand

Anexo 3 - Tabela de Requisitos de Manipulação

REQUISITOS DE MANIPULAÇÃO		
Req.Manipulação		
Identificador	Descrição	Revisor
RM-001	Registar um novo veículo e atualizar as suas características.	Nuno Rebelo
RM-002	Registar um novo cliente e atualizar os seus dados.	Nuno Rebelo
RM-003	Registar um novo posto de aluger e atualizar a sua informação.	Nuno Rebelo
RM-004	Marcar como inativo um veículo, cliente ou posto de aluguer.	Nuno Rebelo
RM-005	Registar um novo aluguer e marcá-lo como iniciado e terminado.	Nuno Rebelo
RM-006	Registar automaticamente um novo pagamento quando um aluguer é iniciado, calculando automaticamente o montante a pagar (fórmula: (DataFim - DataInicio) * TarifaDiaria).	Nuno Rebelo
RM-007	Registar um novo pagamento.	Nuno Rebelo
RM-008	Cancelar um aluguer e/ou pagamento.	Nuno Rebelo
RM-009	Listar os veículos disponíveis para aluguer num dado intervalo de tempo.	Marco Sèvegrand
RM-010	Listar os veículos com umas dadas características técnicas.	Marco Sèvegrand
RM-011	Listar o histórico de aluguer de um cliente.	Marco Sèvegrand
RM-012	Listar o cliente associado a um número de telemóvel.	Marco Sèvegrand
RM-013	Calcular a receita gerada por um posto de aluguer num dado intervalo de tempo.	Marco Sèvegrand
RM-014	Calcular a taxa de ocupaçao de uma categoria de veículo numa dada data.	Marco Sèvegrand

Anexo 4 - Tabela de Requisitos de Controlo

REQUISITOS DE CONTROLO		
Req.Controlo		
Identificador	Descrição	Revisor
RC-001	Um veículo pode apenas estar alugado a um único cliente num dado instante.	Nuno Rebelo
RC-002	Um cliente particular não pode ter mais de 2 alugueres a decorrer em simultâneo.	Nuno Rebelo
RC-003	Apenas utilizadores com o papel de gestor podem registar, modificar e eliminar veículos.	Nuno Rebelo
RC-004	Apenas utilizadores com o papel de gestor podem registar, modificar e eliminar postos de aluguer, eliminar clientes, alugueres e pagamentos.	Nuno Rebelo
RC-005	Deve haver um Admin com acesso total e todos os privilégios.	Nuno Rebelo
RC-006	Qualquer colaborador pode registar e modificar um cliente.	Nuno Rebelo
RC-007	Utilizadores com o papel de colaborador podem registar alugueres e pagamentos, e modificar apenas o estado e a data/hora dos mesmos.	Nuno Rebelo

Anexo 5 - Código do ficheiro *schema.sql*

```
-- Criar ou apagar a base de dados
DROP DATABASE IF EXISTS GreenMotionDB;
CREATE DATABASE GreenMotionDB
        CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

-- Selecionar a base de dados
USE GreenMotionDB;

-- Criar as tabelas
CREATE TABLE PostoAluguer(
        ID INT AUTO_INCREMENT PRIMARY KEY,
```

```
        Nome VARCHAR(45) NOT NULL UNIQUE,
        Morada VARCHAR(200) NOT NULL,
        NumTelefone INT UNSIGNED NOT NULL,
        Email VARCHAR(45) NOT NULL,
        Estado ENUM('Ativo','Inativo') NOT NULL DEFAULT 'Ativo'
);

CREATE TABLE Veiculo(
        ID INT AUTO_INCREMENT PRIMARY KEY,
        PostoAluguer_ID INT NOT NULL,
        Matricula VARCHAR(45) NOT NULL UNIQUE,
        Categoria VARCHAR(45) NOT NULL,
        Marca VARCHAR(45) NOT NULL,
        Modelo VARCHAR(45) NOT NULL,
        TipoMotor VARCHAR(45) NOT NULL,
        NumLugares INT UNSIGNED NOT NULL,
        Autonomia INT UNSIGNED NOT NULL,
        Estado ENUM('Ativo','Inativo') NOT NULL DEFAULT 'Ativo',
        TarifaDiaria DECIMAL(10,2) NOT NULL CHECK (TarifaDiaria >=
        ↪  0),
        FOREIGN KEY (PostoAluguer_ID) REFERENCES PostoAluguer(ID)
);

CREATE TABLE Cliente(
        ID INT AUTO_INCREMENT PRIMARY KEY,
        Nome VARCHAR(45) NOT NULL,
        Tipo ENUM('Individual','Empresa') NOT NULL,
        NIF INT NOT NULL UNIQUE,
        Morada VARCHAR(200) NOT NULL,
        Email VARCHAR(45),
        NumCartaCond INT NOT NULL UNIQUE,
        CategoriaCartaCond VARCHAR(4) NOT NULL,
        ValidadeCartaCond DATE NOT NULL,
        Estado ENUM('Ativo','Inativo') NOT NULL DEFAULT 'Ativo'
);

CREATE TABLE Telemovel(
        Numero INT UNSIGNED NOT NULL,
        Cliente_ID INT NOT NULL,
        PRIMARY KEY (Numero, Cliente_ID),
        FOREIGN KEY (Cliente_ID)
        REFERENCES Cliente(ID)
        ON DELETE CASCADE
);

CREATE TABLE Aluguer(
        ID INT AUTO_INCREMENT PRIMARY KEY,
        Veiculo_ID INT NOT NULL,
```

```
        Cliente_ID INT NOT NULL,
        DataHoraInicio DATETIME NOT NULL,
        DataHoraFim DATETIME NOT NULL,
        Estado ENUM('Agendado','Iniciado','Concluído','Cancelado')
                NOT NULL DEFAULT 'Agendado',
        FOREIGN KEY (Veiculo_ID)
        REFERENCES Veiculo(ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Cliente_ID)
        REFERENCES Cliente(ID)
        ON DELETE CASCADE,
        CHECK (DataHoraFim > DataHoraInicio)
);

CREATE TABLE Pagamento(
        ID INT AUTO_INCREMENT PRIMARY KEY,
        Cliente_ID INT NOT NULL,
        Aluguer_ID INT NOT NULL,
        Montante DECIMAL(10,2) NOT NULL CHECK (Montante >= 0),
        Metodo VARCHAR(45) NOT NULL,
        DataHora DATETIME,
        Estado ENUM
         ↪  ('Pendente','Realizado','Cancelado','Devolvido')
                NOT NULL DEFAULT 'Pendente',
    FOREIGN KEY (Cliente_ID)
                REFERENCES Cliente(ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Aluguer_ID)
        REFERENCES Aluguer(ID)
        ON DELETE CASCADE
);
```

Anexo 6 - Código do ficheiro *privileges_setup.sql*

```
-- Selecionar a base de dados ativa
USE GreenMotionDB;

-- Drop all users
DROP USER IF EXISTS 'colaborador1'@'localhost';
DROP USER IF EXISTS 'colaborador2'@'localhost';
DROP USER IF EXISTS 'gestor1'@'localhost';
DROP USER IF EXISTS 'gestor2'@'localhost';
DROP USER IF EXISTS 'admin1'@'localhost';

-- Drop all roles
DROP ROLE IF EXISTS 'colaborador';
```

```
DROP ROLE IF EXISTS 'gestor';
DROP ROLE IF EXISTS 'admin';

-- Criar papéis
CREATE ROLE 'colaborador', 'gestor', 'admin';

-- Atribuir privilégios aos papéis
GRANT SELECT ON PostoAluguer TO 'colaborador';
GRANT SELECT, UPDATE(Estado) ON Veiculo TO 'colaborador';
GRANT SELECT, INSERT, UPDATE ON Cliente TO 'colaborador';
GRANT SELECT, INSERT, UPDATE, DELETE ON Telemovel TO 'colaborador';
GRANT SELECT, INSERT, UPDATE(Estado) ON Aluguer TO 'colaborador';
GRANT SELECT, INSERT, UPDATE(DataHora, Estado) ON Pagamento TO
 ↪  'colaborador';

GRANT SELECT, INSERT, UPDATE, DELETE ON PostoAluguer TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Veiculo TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Cliente TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Telemovel TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Aluguer TO 'gestor';
GRANT SELECT, INSERT, UPDATE, DELETE ON Pagamento TO 'gestor';

GRANT ALL PRIVILEGES ON GreenMotionDB.* TO 'admin' WITH GRANT
 ↪  OPTION;
GRANT 'colaborador', 'gestor' TO 'admin' WITH ADMIN OPTION;

-- Criar utilizadores
CREATE USER 'colaborador1'@'localhost' IDENTIFIED BY 'password1';
CREATE USER 'colaborador2'@'localhost' IDENTIFIED BY 'password2';
CREATE USER 'gestor1'@'localhost' IDENTIFIED BY 'password3';
CREATE USER 'gestor2'@'localhost' IDENTIFIED BY 'password4';
CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'password5';

-- Atribuir papéis aos utilizadores
GRANT 'colaborador' TO 'colaborador1'@'localhost';
GRANT 'colaborador' TO 'colaborador2'@'localhost';
GRANT 'gestor' TO 'gestor1'@'localhost';
GRANT 'gestor' TO 'gestor2'@'localhost';
GRANT 'admin' TO 'admin1'@'localhost';

-- Definir papéis padrão
SET DEFAULT ROLE 'colaborador' TO 'colaborador1'@'localhost';
SET DEFAULT ROLE 'colaborador' TO 'colaborador2'@'localhost';
SET DEFAULT ROLE 'gestor' TO 'gestor1'@'localhost';
SET DEFAULT ROLE 'gestor' TO 'gestor2'@'localhost';
SET DEFAULT ROLE 'admin' TO 'admin1'@'localhost';

-- Atualizar privilégios
```

```
FLUSH PRIVILEGES;
```

Anexo 7 - Código do ficheiro *privileges_setup.sql*

```
-- Desativar verificações de chave estrangeira temporariamente
SET FOREIGN_KEY_CHECKS = 0;

-- Limpar tabelas na ordem correta
TRUNCATE TABLE Pagamento;
TRUNCATE TABLE Aluguer;
TRUNCATE TABLE Telemovel;
TRUNCATE TABLE Cliente;
TRUNCATE TABLE Veiculo;
TRUNCATE TABLE PostoAluguer;

-- Reativar verificações de chave estrangeira
SET FOREIGN_KEY_CHECKS = 1;

-- Povoamento das Tabelas

-- Inserir postos de aluguer (3 exemplos)
INSERT INTO PostoAluguer VALUES
(1, 'Posto Central', 'Rua da Liberdade, 101', 912345678,
↪  'central@greenmotion.pt', 'Ativo'),
(2, 'Posto Norte', 'Av. do Minho, 99', 913456789,
↪  'norte@greenmotion.pt', 'Ativo'),
(3, 'Posto Sul', 'Rua Algarve, 22', 914567890,
↪  'sul@greenmotion.pt', 'Ativo');

-- Inserir veículos (15 exemplos)
INSERT INTO Veiculo VALUES
(1, 1, 'AA-00-01', 'SUV', 'Tesla', 'Model X', 'Elétrico', 5, 500,
↪  'Ativo', 89.99),
(2, 1, 'BB-11-22', 'Citadino', 'Renault', 'ZOE', 'Elétrico', 4,
↪  300, 'Ativo', 49.99),
(3, 1, 'CC-33-44', 'Carrinha', 'Volkswagen', 'ID.Buzz', 'Elétrico',
↪  7, 400, 'Ativo', 79.99),
(4, 2, 'DD-44-55', 'Desportivo', 'Porsche', 'Taycan', 'Elétrico',
↪  4, 450, 'Inativo', 129.99),
(5, 2, 'EE-55-66', 'SUV', 'BMW', 'iX', 'Híbrido', 5, 480,
↪  'Inativo', 99.99),
(6, 2, 'FF-66-77', 'Citadino', 'Nissan', 'Leaf', 'Elétrico', 5,
↪  350, 'Ativo', 59.99),
(7, 3, 'GG-77-88', 'Familiar', 'Toyota', 'Mirai', 'Hidrogénio', 5,
↪  550, 'Ativo', 74.99),
```

```
(8, 3, 'HH-88-99', 'Comercial', 'Mercedes', 'eSprinter',
↪  'Elétrico', 3, 300, 'Ativo', 69.99),
(9, 1, 'II-99-00', 'Familiar', 'Hyundai', 'Ioniq 5', 'Elétrico', 5,
↪  480, 'Ativo', 84.99),
(10, 2, 'JJ-10-20', 'Comercial', 'Peugeot', 'e-Expert', 'Elétrico',
↪  3, 320, 'Ativo', 64.99),
(11, 3, 'KK-21-31', 'SUV', 'Kia', 'EV6', 'Elétrico', 5, 510,
↪  'Ativo', 92.99),
(12, 3, 'LL-32-42', 'Carrinha', 'Ford', 'E-Transit', 'Elétrico', 7,
↪  420, 'Inativo', 77.99),
(13, 2, 'MM-43-53', 'Citadino', 'Fiat', '500e', 'Elétrico', 4, 280,
↪  'Ativo', 44.99),
(14, 1, 'NN-54-64', 'Desportivo', 'Audi', 'e-tron GT', 'Elétrico',
↪  4, 430, 'Ativo', 124.99),
(15, 3, 'OO-65-75', 'SUV', 'Volvo', 'XC40 Recharge', 'Elétrico', 5,
↪  400, 'Ativo', 88.99);

-- Inserir clientes (15 exemplos)
INSERT INTO Cliente VALUES
(1, 'João Silva', 'Individual', 123456789, 'Rua das Flores, 45',
↪  'joao@gmail.com',
987654321, 'B', '2026-12-31', 'Ativo'),
(2, 'Empresa XPTO', 'Empresa', 509876543, 'Parque Industrial, Lote
↪  3',
'empresa@xpto.pt', 112233445, 'C1', '2030-05-15', 'Ativo'),
(3, 'Maria Rocha', 'Individual', 234567891, 'Av. Mar, 87',
↪  'maria@hotmail.com',
223344556, 'B', '2028-03-20', 'Ativo'),
(4, 'Carlos Mendes', 'Individual', 345678912, 'Rua das Oliveiras,
↪  12',
'carlos@sapo.pt', 334455667, 'A', '2027-11-30', 'Ativo'),
(5, 'Tech Solutions Lda', 'Empresa', 501234567, 'Edifício Moderno,
↪  Piso 3',
'contas@techsol.pt', 445566778, 'C2', '2031-08-22', 'Ativo'),
(6, 'Ana Sousa', 'Individual', 456789123, 'Travessa do Sol, 7',
'ana.sousa@outlook.com', 556677889, 'B', '2029-01-10', 'Inativo'),
(7, 'Green Energy SA', 'Empresa', 502345678, 'Parque Empresarial,
↪  Bloco B',
'geral@greenenergy.pt', 667788990, 'C1', '2032-04-18', 'Ativo'),
(8, 'Pedro Almeida', 'Individual', 567891234, 'Av. Central, 105',
'pedro.almeida@gmail.com', 778899001, 'A', '2027-09-15', 'Ativo'),
(9, 'Build Construction', 'Empresa', 503456789, 'Zona Industrial,
↪  Armazém 5',
'financeiro@buildcon.pt', 889900112, 'C2', '2033-07-01', 'Ativo'),
(10, 'Sofia Costa', 'Individual', 678912345, 'Rua da Praia, 18',
'sofia.costa@yahoo.com', 990011223, 'B', '2028-05-25', 'Ativo'),
(11, 'Ricardo Pinto', 'Individual', 789123456, 'Rua Nova, 15',
'ricardo.pinto@gmail.com', 101112131, 'B', '2027-06-20', 'Ativo'),
```

```
(12, 'LogiTrans Lda', 'Empresa', 504567890, 'Estrada Nacional,
↪  200',
'contacto@logitrans.pt', 141516171, 'C2', '2034-01-30', 'Ativo'),
(13, 'Isabel Faria', 'Individual', 891234567, 'Av. das Amoreiras,
↪  33',
'isabel.faria@hotmail.com', 181920212, 'A', '2029-10-10', 'Ativo'),
(14, 'Pedro Lopes', 'Individual', 912345678, 'Rua das Laranjeiras,
↪  8',
'pedro.lopes@sapo.pt', 222324252, 'B', '2028-12-31', 'Inativo'),
(15, 'Smart Build SA', 'Empresa', 505678901, 'Rua do Progresso,
↪  50',
'info@smartbuild.pt', 262728293, 'C1', '2032-11-05', 'Ativo');

-- Inserir telemóveis (20 exemplos)
INSERT INTO Telemovel VALUES
(912222333, 1),
(935555666, 2),
(968888999, 3),
(913333444, 4),
(936666777, 5),
(969999000, 6),
(914444555, 7),
(937777888, 8),
(915555666, 9),
(938888999, 10),
(916666777, 1),
(939999000, 5),
(917111222, 11),
(918222333, 12),
(919333444, 13),
(920444555, 14),
(921555666, 15),
(922666777, 12),
(923777888, 13),
(924888999, 14);

-- Inserir alugueres (30 exemplos)
INSERT INTO Aluguer VALUES
(1, 1, 1, '2025-04-20 10:00:00', '2025-04-21 10:00:00',
↪  'Concluído'),
(2, 2, 3, '2025-04-18 09:00:00', '2025-04-19 09:00:00',
↪  'Iniciado'),
(3, 3, 2, '2025-04-15 12:00:00', '2025-04-16 12:00:00',
↪  'Concluído'),
(4, 4, 5, '2025-04-22 14:00:00', '2025-04-25 14:00:00',
↪  'Agendado'),
(5, 5, 7, '2025-04-19 08:00:00', '2025-04-20 08:00:00',
↪  'Cancelado'),
```

```
(6, 6, 1, '2025-04-25 16:00:00', '2025-04-26 16:00:00',
↪  'Agendado'),
(7, 7, 4, '2025-04-17 11:00:00', '2025-04-18 11:00:00',
↪  'Concluído'),
(8, 8, 6, '2025-04-23 09:00:00', '2025-04-24 09:00:00',
↪  'Cancelado'),
(9, 9, 11, '2025-05-01 10:00:00', '2025-05-03 10:00:00',
↪  'Agendado'),
(10, 10, 12, '2025-05-04 09:00:00', '2025-05-06 09:00:00',
↪  'Concluído'),
(11, 11, 13, '2025-05-07 11:00:00', '2025-05-08 11:00:00',
↪  'Iniciado'),
(12, 12, 14, '2025-05-10 14:00:00', '2025-05-12 14:00:00',
↪  'Cancelado'),
(13, 13, 15, '2025-05-13 08:00:00', '2025-05-15 08:00:00',
↪  'Concluído'),
(14, 14, 1, '2025-05-16 12:00:00', '2025-05-18 12:00:00',
↪  'Agendado'),
(15, 15, 2, '2025-05-20 09:00:00', '2025-05-21 09:00:00',
↪  'Concluído'),
(16, 1, 9, '2025-06-01 09:00:00', '2025-06-03 09:00:00',
↪  'Agendado'),
(17, 2, 10, '2025-06-02 10:00:00', '2025-06-04 10:00:00',
↪  'Concluído'),
(18, 3, 11, '2025-06-05 11:00:00', '2025-06-07 11:00:00',
↪  'Cancelado'),
(19, 4, 12, '2025-06-08 08:30:00', '2025-06-10 08:30:00',
↪  'Iniciado'),
(20, 5, 13, '2025-06-09 14:00:00', '2025-06-11 14:00:00',
↪  'Agendado'),
(21, 6, 14, '2025-06-12 15:00:00', '2025-06-14 15:00:00',
↪  'Concluído'),
(22, 7, 15, '2025-06-15 09:30:00', '2025-06-17 09:30:00',
↪  'Cancelado'),
(23, 8, 9, '2025-06-18 10:00:00', '2025-06-20 10:00:00',
↪  'Agendado'),
(24, 9, 10, '2025-06-21 11:00:00', '2025-06-23 11:00:00',
↪  'Concluído'),
(25, 10, 11, '2025-06-24 08:00:00', '2025-06-26 08:00:00',
↪  'Iniciado'),
(26, 11, 12, '2025-06-27 13:00:00', '2025-06-29 13:00:00',
↪  'Agendado'),
(27, 12, 13, '2025-06-30 14:30:00', '2025-07-02 14:30:00',
↪  'Concluído'),
(28, 13, 14, '2025-07-03 09:00:00', '2025-07-05 09:00:00',
↪  'Cancelado'),
(29, 14, 15, '2025-07-06 10:15:00', '2025-07-08 10:15:00',
↪  'Agendado'),
```

```
(30, 15, 1, '2025-07-09 11:45:00', '2025-07-11 11:45:00',
↪  'Concluído');

-- Inserir pagamentos (35 exemplos)
INSERT INTO Pagamento VALUES
(1, 1, 1, 89.99, 'MBWay', '2025-04-20 09:55:00', 'Realizado'),
(2, 3, 2, 49.99, 'Cartão Crédito', '2025-04-18 08:50:00',
↪  'Realizado'),
(3, 2, 3, 79.99, 'Transferência', '2025-04-15 11:30:00',
↪  'Pendente'),
(4, 4, 5, 289.97, 'Cartão Débito', '2025-04-22 13:45:00',
↪  'Realizado'),
(5, 5, 7, 74.99, 'MBWay', '2025-04-19 07:50:00', 'Realizado'),
(6, 6, 1, 89.99, 'PayPal', '2025-04-25 15:30:00', 'Pendente'),
(7, 7, 4, 129.99, 'Cartão Crédito', '2025-04-17 10:45:00',
↪  'Realizado'),
(8, 8, 6, 59.99, 'Transferência', '2025-04-23 08:20:00',
↪  'Cancelado'),
(9, 1, 1, 10.00, 'MBWay', '2025-04-20 10:30:00', 'Realizado'),
(10, 5, 7, 15.00, 'Cartão Débito', '2025-04-19 12:15:00',
↪  'Realizado'),
(11, 9, 11, 169.98, 'Cartão Crédito', '2025-05-01 09:50:00',
↪  'Pendente'),
(12, 10, 12, 129.98, 'MBWay', '2025-05-04 08:55:00', 'Realizado'),
(13, 11, 13, 92.99, 'Transferência', '2025-05-07 10:50:00',
↪  'Realizado'),
(14, 12, 14, 77.99, 'Cartão Débito', '2025-05-10 13:45:00',
↪  'Cancelado'),
(15, 13, 15, 88.99, 'PayPal', '2025-05-13 07:50:00', 'Realizado'),
(16, 14, 1, 84.99, 'MBWay', '2025-05-16 11:30:00', 'Pendente'),
(17, 9, 11, 10.00, 'Cartão Débito', '2025-05-01 10:30:00',
↪  'Realizado'),
(18, 13, 15, 15.00, 'Transferência', '2025-05-13 12:15:00',
↪  'Realizado'),
(19, 1, 9, 84.99, 'MBWay', '2025-06-01 08:50:00', 'Pendente'),
(20, 2, 10, 64.99, 'Cartão Crédito', '2025-06-02 09:45:00',
↪  'Realizado'),
(21, 3, 11, 92.99, 'Transferência', '2025-06-05 10:30:00',
↪  'Cancelado'),
(22, 4, 12, 77.99, 'Cartão Débito', '2025-06-08 07:55:00',
↪  'Realizado'),
(23, 5, 13, 44.99, 'PayPal', '2025-06-09 13:20:00', 'Realizado'),
(24, 6, 14, 124.99, 'MBWay', '2025-06-12 14:10:00', 'Pendente'),
(25, 7, 15, 88.99, 'Cartão Crédito', '2025-06-15 08:40:00',
↪  'Realizado'),
(26, 8, 9, 74.99, 'Transferência', '2025-06-18 09:00:00',
↪  'Cancelado'),
```

```
(27, 9, 10, 64.99, 'Cartão Débito', '2025-06-21 10:15:00',
↪  'Realizado'),
(28, 10, 11, 92.99, 'MBWay', '2025-06-24 07:30:00', 'Realizado'),
(29, 11, 12, 77.99, 'PayPal', '2025-06-27 12:45:00', 'Pendente'),
(30, 12, 13, 44.99, 'Cartão Crédito', '2025-06-30 13:50:00',
↪  'Realizado'),
(31, 13, 14, 124.99, 'Transferência', '2025-07-03 08:20:00',
↪  'Cancelado'),
(32, 14, 15, 88.99, 'MBWay', '2025-07-06 09:35:00', 'Realizado'),
(33, 15, 1, 89.99, 'Cartão Débito', '2025-07-09 11:00:00',
↪  'Realizado'),
(34, 1, 9, 84.99, 'PayPal', '2025-06-01 09:10:00', 'Realizado'),
(35, 2, 10, 64.99, 'MBWay', '2025-06-02 10:00:00', 'Pendente');
```

Anexo 8 - Código do ficheiro *import_script.py*

```
import psycopg2
import mysql.connector
import pandas as pd
import json
from datetime import datetime

# Database configurations
POSTGRES_CONFIG = {'host': 'localhost',
                   'database': 'clientesbd',
                   'user': 'postgres',
                   'password': 'postgres'}
MYSQL_CONFIG = {'host': 'localhost',
                'database': 'GreenMotionDB',
                'user': 'root',
                'password': 'root'}
DATA_FILES = {
    'postos_csv': 'postosdealuguer.csv',
    'veiculos_csv': 'veiculos.csv',
    'alugueres_csv': 'alugueres.csv',
    'pagamentos_json': 'pagamentos.json'
}

# Database connection helpers
def get_mysql_connection(): return
↪  mysql.connector.connect(**MYSQL_CONFIG)
def get_postgres_connection(): return
↪  psycopg2.connect(**POSTGRES_CONFIG)

# Data insertion helpers
def insert_with_id_mapping(table_name, columns, data_rows):
```

```
    conn = get_mysql_connection()
    cursor = conn.cursor()
    insert_columns = columns[1:]
    placeholders = ', '.join(['%s'] * len(insert_columns))
    query = f"INSERT INTO {table_name} ({',
    ↪  '.join(insert_columns)}) VALUES ({placeholders})"
    id_mapping = {}
    successful = 0
    total = len(data_rows)

    for row in data_rows:
        old_id = row[0]
        values_to_insert = row[1:]
        try:
            cursor.execute(query, values_to_insert)
            conn.commit()
            cursor.execute("SELECT LAST_INSERT_ID()")
            id_mapping[old_id] = cursor.fetchone()[0]
            successful += 1
        except Exception:
            conn.rollback()

    cursor.close()
    conn.close()
    print(f"{successful}/{total} records inserted into
    ↪  {table_name}")
    return id_mapping

def insert_simple_data(table_name, columns, data_rows):
    if not data_rows:
        print(f"0/0 records inserted into {table_name}")
        return

    conn = get_mysql_connection()
    cursor = conn.cursor()
    placeholders = ', '.join(['%s'] * len(columns))
    query = f"INSERT INTO {table_name} ({', '.join(columns)})
    ↪  VALUES ({placeholders})"
    total = len(data_rows)

    try:
        cursor.executemany(query, data_rows)
        conn.commit()
        successful = cursor.rowcount
    except Exception:
        conn.rollback()
        successful = 0
    finally:
```

```
        cursor.close()
        conn.close()

    print(f"{successful}/{total} records inserted into
     ↪  {table_name}")

# Migration functions
def migrate_postos_from_csv():
    df = pd.read_csv(DATA_FILES['postos_csv'])
    data_to_insert = [(int(r['ID']), r['Nome'], r['Morada'],
     ↪  int(r['NumTelefone']),
                      r['Email'], r['Estado'].capitalize()) for _,
                       ↪  r in df.iterrows()]
    columns = ['ID', 'Nome', 'Morada', 'NumTelefone', 'Email',
     ↪  'Estado']
    return insert_with_id_mapping('PostoAluguer', columns,
     ↪  data_to_insert)

def migrate_clientes_from_postgres():
    conn = get_postgres_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT ID, Nome, Tipo, NIF, Morada, Email,
     ↪  NumCartaCond,
        CategoriaCartaCond, ValidadeCartaCond, Estado FROM
         ↪  Cliente")
    data_to_insert = [(r[0], r[1], r[2].capitalize(), r[3], r[4],
     ↪  r[5], r[6],
                      r[7].upper() if r[7] else None, r[8],
                       ↪  r[9].capitalize()) for r in
                       ↪  cursor.fetchall()]
    cursor.close()
    conn.close()
    columns = ['ID', 'Nome', 'Tipo', 'NIF', 'Morada', 'Email',
        'NumCartaCond', 'CategoriaCartaCond', 'ValidadeCartaCond',
         ↪  'Estado']
    return insert_with_id_mapping('Cliente', columns,
     ↪  data_to_insert)

def migrate_phone_numbers_from_postgres(client_id_mapping):
    conn = get_postgres_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT Numero, Cliente_ID FROM Telemovel")
    data_to_insert = [(r[0], client_id_mapping.get(r[1]))
                for r in cursor.fetchall() if r[1] in
                 ↪  client_id_mapping]
    cursor.close()
    conn.close()
```

```
    insert_simple_data('Telemovel', ['Numero', 'Cliente_ID'],
    ↪  data_to_insert)

def migrate_vehicles_from_csv(station_id_mapping):
    df = pd.read_csv(DATA_FILES['veiculos_csv'])
    data_to_insert = []
    for _, r in df.iterrows():
        new_station_id =
        ↪  station_id_mapping.get(int(r['PostoAluguer_ID']))
        if new_station_id:
            data_to_insert.append((int(r['ID']), new_station_id,
            ↪  r['Matricula'], r['Categoria'], r['Marca'],
                                r['Modelo'], r['TipoMotor'],
                                ↪  int(r['NumLugares']),
                                ↪  int(r['Autonomia']),
                                r.get('Estado',
                                ↪  'Ativo').capitalize(),
                                ↪  float(r['TarifaDiaria'])))
    columns = ['ID', 'PostoAluguer_ID', 'Matricula', 'Categoria',
    ↪  'Marca', 'Modelo', 'TipoMotor',
              'NumLugares', 'Autonomia', 'Estado', 'TarifaDiaria']
    return insert_with_id_mapping('Veiculo', columns,
    ↪  data_to_insert)

def migrate_rentals_from_csv(vehicle_id_mapping,
↪  client_id_mapping):
    df = pd.read_csv(DATA_FILES['alugueres_csv'])
    rental_id_mapping = {}
    successful = 0
    total = len(df)
    conn = get_mysql_connection()
    cursor = conn.cursor()

    for _, r in df.iterrows():
        old_id = int(r['ID'])
        new_vehicle_id =
        ↪  vehicle_id_mapping.get(int(r['Veiculo_ID']))
        new_client_id = client_id_mapping.get(int(r['Cliente_ID']))
        if not new_vehicle_id or not new_client_id:
            continue

        try:
            cursor.execute(
                "INSERT INTO Aluguer (Veiculo_ID, Cliente_ID,
                    DataHoraInicio, DataHoraFim, Estado)
                    VALUES (%s, %s, %s, %s, %s)",
                (new_vehicle_id, new_client_id,
                    datetime.strptime(r['DataHoraInicio'],
```

```
                    '%Y-%m-%d %H:%M:%S'),
                datetime.strptime(r['DataHoraFim'],
                    '%Y-%m-%d %H:%M:%S'), r.get('Estado',
                    'Agendado').capitalize())
            )
            conn.commit()
            cursor.execute("SELECT LAST_INSERT_ID()")
            rental_id_mapping[old_id] = cursor.fetchone()[0]
            successful += 1
        except Exception:
            conn.rollback()

    cursor.close()
    conn.close()
    print(f"{successful}/{total} records inserted into Aluguer")
    return rental_id_mapping

def migrate_payments_from_json(client_id_mapping,
 ↪  rental_id_mapping):
    with open(DATA_FILES['pagamentos_json'], 'r') as file:
        payments = json.load(file)

    data_to_insert = []
    for p in payments:
        new_client_id = client_id_mapping.get(int(p['Cliente_ID']))
        new_rental_id = rental_id_mapping.get(int(p['Aluguer_ID']))
        if new_client_id and new_rental_id:
            payment_datetime =
             ↪  datetime.fromisoformat(p['DataHora']) if
             ↪  p.get('DataHora') else None
            data_to_insert.append((new_client_id, new_rental_id,
             ↪  float(p['Montante']),
                                   p['Metodo'], payment_datetime,
                                    ↪  p.get('Estado',
                                    ↪  'Pendente').capitalize()))

    insert_simple_data('Pagamento', ['Cliente_ID',
    'Aluguer_ID', 'Montante', 'Metodo',
    'DataHora', 'Estado'], data_to_insert)

# Main execution
def run_migration():
    print("Starting database migration...")
    print("=" * 30)
    try:
        station_id_mapping = migrate_postos_from_csv()
        client_id_mapping = migrate_clientes_from_postgres()
        migrate_phone_numbers_from_postgres(client_id_mapping)
```

```
        vehicle_id_mapping =
         ↪  migrate_vehicles_from_csv(station_id_mapping)
        rental_id_mapping =
         ↪  migrate_rentals_from_csv(vehicle_id_mapping,
         ↪  client_id_mapping)
        migrate_payments_from_json(client_id_mapping,
         ↪  rental_id_mapping)
        print("=" * 30)
        print("Migration completed successfully!")
    except Exception as e:
        print("=" * 30)
        print(f"Migration failed: {str(e)}")

if __name__ == "__main__":
    run_migration()
```

Anexo 9 - Código do ficheiro *procedures_functions_views.sql*

```
-- ======================================
-- FICHEIRO: procedures_functions_triggers.sql
-- Descrição: Procedures, Funções e Views relacionadas com os
 ↪  Requisitos RM-001 a RM-014
-- ======================================

DELIMITER $$

-- RM-001: Inserir cliente
CREATE PROCEDURE InserirCliente(
    IN pNome VARCHAR(45),
    IN pTipo ENUM('Individual','Empresa'),
    IN pNIF INT,
    IN pMorada VARCHAR(200),
    IN pEmail VARCHAR(45),
    IN pNumCartaCond INT,
    IN pCategoriaCarta VARCHAR(4),
    IN pValidadeCarta DATE
)
BEGIN
    INSERT INTO Cliente (Nome, Tipo, NIF, Morada, Email,
     ↪  NumCartaCond, CategoriaCartaCond, ValidadeCartaCond,
     ↪  Estado)
    VALUES (pNome, pTipo, pNIF, pMorada, pEmail, pNumCartaCond,
     ↪  pCategoriaCarta, pValidadeCarta, 'Ativo');
END $$
```

```
CREATE PROCEDURE InserirTelemovel(IN ClienteID INT, IN Numero INT
 ↪  UNSIGNED)
BEGIN
    INSERT INTO Telemovel (Cliente_ID, Numero) VALUES (ClienteID,
     ↪  Numero);
END $$

-- RM-002: Inserir veículo
CREATE PROCEDURE InserirVeiculo(
    IN pPostoID INT, IN pMatricula VARCHAR(45), IN pCategoria
     ↪  VARCHAR(45),
    IN pMarca VARCHAR(45), IN pModelo VARCHAR(45), IN pTipoMotor
     ↪  VARCHAR(45),
    IN pNumLugares INT, IN pAutonomia INT, IN pTarifaDiaria
     ↪  DECIMAL(10,2)
)
BEGIN
    INSERT INTO Veiculo (PostoAluguer_ID, Matricula, Categoria,
     ↪  Marca, Modelo, TipoMotor, NumLugares, Autonomia,
     ↪  TarifaDiaria, Estado)
    VALUES (pPostoID, pMatricula, pCategoria, pMarca, pModelo,
     ↪  pTipoMotor, pNumLugares, pAutonomia, pTarifaDiaria,
     ↪  'Ativo');
END $$

-- RM-003: Criar aluguer com transação e criação automática de
 ↪  pagamento
CREATE PROCEDURE CriarAluguer(
    IN pVeiculoID INT, IN pClienteID INT, IN pInicio DATETIME, IN
     ↪  pFim DATETIME
)
BEGIN
    DECLARE AluguerID INT;
    DECLARE Tarifa DECIMAL(10,2);
    DECLARE Dias INT;
    DECLARE Montante DECIMAL(10,2);
    START TRANSACTION;
    INSERT INTO Aluguer (Veiculo_ID, Cliente_ID, DataHoraInicio,
     ↪  DataHoraFim, Estado)
    VALUES (pVeiculoID, pClienteID, pInicio, pFim, 'Agendado');
    SET AluguerID = LAST_INSERT_ID();
    SELECT TarifaDiaria INTO Tarifa FROM Veiculo WHERE ID =
     ↪  pVeiculoID;
    SET Dias = DATEDIFF(pFim, pInicio);
    SET Montante = Tarifa * Dias;
    INSERT INTO Pagamento (Cliente_ID, Aluguer_ID, Montante,
     ↪  Metodo, DataHora, Estado)
```

```
    VALUES (pClienteID, AluguerID, Montante, 'Multibanco', NOW(),
     ↪  'Pendente');
    COMMIT;
END $$

-- RM-004: Atualizar cliente
CREATE PROCEDURE AtualizarCliente(IN ClienteID INT, IN pNome
 ↪  VARCHAR(45), IN pMorada VARCHAR(200), IN pEmail VARCHAR(45))
BEGIN
    UPDATE Cliente SET Nome = pNome, Morada = pMorada, Email =
     ↪  pEmail WHERE ID = ClienteID;
END $$

-- RM-005: Cancelar aluguer
CREATE PROCEDURE CancelarAluguer(IN AluguerID INT)
BEGIN
    UPDATE Aluguer SET Estado = 'Cancelado' WHERE ID = AluguerID
     ↪  AND Estado = 'Agendado';
END $$

-- RM-006: Remover veículo
CREATE PROCEDURE RemoverVeiculo(IN VeiculoID INT)
BEGIN
    DELETE FROM Veiculo WHERE ID = VeiculoID;
END $$

-- RM-007: Remover cliente
CREATE PROCEDURE RemoverCliente(IN ClienteID INT)
BEGIN
    DELETE FROM Cliente WHERE ID = ClienteID;
END $$

-- RM-008: Alterar estado do aluguer
CREATE PROCEDURE AlterarEstadoAluguer(IN AluguerID INT, IN
 ↪  NovoEstado ENUM('Agendado','Iniciado','Concluído','Cancelado'))
BEGIN
    UPDATE Aluguer SET Estado = NovoEstado WHERE ID = AluguerID;
END $$

-- RM-009: Veículos disponíveis
CREATE PROCEDURE VeiculosDisponiveis(IN DataInicio DATETIME, IN
 ↪  DataFim DATETIME)
BEGIN
    SELECT * FROM Veiculo v
    WHERE v.Estado = 'Ativo' AND v.ID NOT IN (
        SELECT a.Veiculo_ID FROM Aluguer a
        WHERE a.Estado IN ('Iniciado', 'Agendado')
```

```
        AND (a.DataHoraInicio <= DataFim AND a.DataHoraFim >=
         ↪  DataInicio)
    );
END $$

-- RM-010: Veículos por características
CREATE PROCEDURE VeiculosPorCaracteristicas(
    IN pCategoria VARCHAR(45), IN pMarca VARCHAR(45), IN pModelo
     ↪  VARCHAR(45),
    IN pTipoMotor VARCHAR(45), IN pNumLugares INT
)
BEGIN
    SELECT * FROM Veiculo
    WHERE Estado = 'Ativo'
    AND (Categoria = pCategoria OR pCategoria IS NULL)
    AND (Marca = pMarca OR pMarca IS NULL)
    AND (Modelo = pModelo OR pModelo IS NULL)
    AND (TipoMotor = pTipoMotor OR pTipoMotor IS NULL)
    AND (NumLugares = pNumLugares OR pNumLugares IS NULL);
END $$

-- RM-011: Histórico de aluguer
CREATE PROCEDURE HistoricoAluguerCliente(IN ClienteID INT)
BEGIN
    SELECT a.*, v.Marca, v.Modelo
    FROM Aluguer a JOIN Veiculo v ON a.Veiculo_ID = v.ID
    WHERE a.Cliente_ID = ClienteID
    ORDER BY a.DataHoraInicio DESC;
END $$

-- RM-012: Obter cliente por telemóvel
CREATE PROCEDURE ClientePorTelemovel(IN NumeroTelemovel INT)
BEGIN
    SELECT c.* FROM Cliente c JOIN Telemovel t ON c.ID =
     ↪  t.Cliente_ID
    WHERE t.Numero = NumeroTelemovel;
END $$

-- RM-013: Receita do posto
CREATE PROCEDURE ReceitaPosto(IN PostoID INT, IN DataInicio
 ↪  DATETIME, IN DataFim DATETIME)
BEGIN
    SELECT pa.ID AS PostoID, pa.Nome, SUM(p.Montante) AS
     ↪  ReceitaTotal
    FROM Pagamento p
    JOIN Aluguer a ON p.Aluguer_ID = a.ID
    JOIN Veiculo v ON a.Veiculo_ID = v.ID
    JOIN PostoAluguer pa ON v.PostoAluguer_ID = pa.ID
```

```
    WHERE p.Estado = 'Realizado'
    AND p.DataHora BETWEEN DataInicio AND DataFim
    AND pa.ID = PostoID
    GROUP BY pa.ID, pa.Nome;
END $$

DELIMITER $$

CREATE FUNCTION TaxaOcupacao(CategoriaVeiculo VARCHAR(45), DataRef
 ↪  DATETIME)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE Total INT DEFAULT 0;
    DECLARE Ocupados INT DEFAULT 0;
    DECLARE Taxa DECIMAL(5,2);

    -- Total de veículos ativos da categoria
    SELECT COUNT(*) INTO Total
    FROM Veiculo
    WHERE Categoria = CategoriaVeiculo AND Estado = 'Ativo';

    -- Veículos ocupados naquele instante, considerando qualquer
     ↪  aluguer NÃO cancelado
    SELECT COUNT(DISTINCT v.ID) INTO Ocupados
    FROM Veiculo v
    JOIN Aluguer a ON v.ID = a.Veiculo_ID
    WHERE v.Categoria = CategoriaVeiculo
      AND v.Estado = 'Ativo'
      AND a.DataHoraInicio <= DataRef
      AND a.DataHoraFim >= DataRef
      AND a.Estado <> 'Cancelado';

    SET Taxa = IF(Total = 0, 0, (Ocupados * 100.0) / Total);
    RETURN Taxa;
END$$

DELIMITER ;


DELIMITER $$

-- VIEW: VeiculosComDetalhes
-- Objetivo: Fornece detalhes básicos dos veículos ativos com o
 ↪  nome do posto de aluguer
CREATE OR REPLACE VIEW VeiculosComDetalhes AS
SELECT v.ID, v.Matricula, v.Marca, v.Modelo, v.Categoria, p.Nome AS
 ↪  Posto
```

```
FROM Veiculo v
JOIN PostoAluguer p ON v.PostoAluguer_ID = p.ID
WHERE v.Estado = 'Ativo';
$$

DELIMITER ;
```

Anexo 10 - Código do ficheiro *triggers.sql*

```
-- ======================================
-- GATILHOS (TRIGGERS)
-- Requisitos adicionais do ponto 5.8: Garantia de integridade e
 ↪  validação
-- ======================================

DELIMITER $$
CREATE TRIGGER prevent_double_booking
BEFORE INSERT ON Aluguer
FOR EACH ROW
BEGIN
  IF EXISTS (
    SELECT 1 FROM Aluguer
    WHERE Veiculo_ID = NEW.Veiculo_ID
      AND Estado IN ('Agendado', 'Iniciado')
      AND (
        (NEW.DataHoraInicio BETWEEN DataHoraInicio AND DataHoraFim)
        OR
        (NEW.DataHoraFim BETWEEN DataHoraInicio AND DataHoraFim)
        OR
        (DataHoraInicio BETWEEN NEW.DataHoraInicio AND
         ↪  NEW.DataHoraFim)
      )
  ) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Vehicle already booked for this period!';
  END IF;
END$$

CREATE TRIGGER prevent_expired_license
BEFORE INSERT ON Aluguer
FOR EACH ROW
BEGIN
  DECLARE validade DATE;
  SELECT ValidadeCartaCond INTO validade FROM Cliente WHERE ID =
   ↪  NEW.Cliente_ID;
  IF validade < CURDATE() THEN
```

```
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot rent: Driver\'s license expired!';
  END IF;
END$$
DELIMITER ;
```

Anexo 11 - Código do ficheiro *queries_call.sql*

```
-- ======================================
-- FICHEIRO: queries_calls.sql
-- Descrição: Exemplos de consultas, chamadas de procedures e
 ↪  funções com comentários
-- Relacionado aos requisitos RM-001 a RM-014
-- ======================================

-- ====================================================
-- Visualização dos registos de todas as tabelas
-- ====================================================
USE GreenMotionDB;
SELECT * FROM PostoAluguer;
SELECT * FROM Veiculo;
SELECT * FROM Cliente;
SELECT * FROM Telemovel;
SELECT * FROM Aluguer;
SELECT * FROM Pagamento;

-- ====================================================
-- RM-001: Inserir novo cliente e número de telemóvel
-- ====================================================
CALL InserirCliente(
    'João Silva', 'Individual', 120926709, 'Rua Central 45',
    'joao@email.com', 4411100, 'B', '2028-12-31'
);

CALL InserirTelemovel(1, 912395678);

-- ====================================================
-- RM-002: Inserir novo veículo
-- ====================================================
CALL InserirVeiculo(
    1, '00-AA-11', 'SUV', 'Tesla', 'Model X',
    'Elétrico', 5, 450, 95.50
);

-- ====================================================
-- RM-003: Criar novo aluguer com pagamento automático (transação)
```

```
-- ====================================================
CALL CriarAluguer(
    4, 1, '2025-06-01 10:00:00', '2025-06-05 10:00:00'
);

-- ====================================================
-- RM-004: Atualizar dados do cliente
-- ====================================================
CALL AtualizarCliente(
    1, 'João Silva Neto', 'Avenida Nova 10', 'joao.neto@email.com'
);

-- ====================================================
-- RM-005: Cancelar aluguer
-- ====================================================
CALL CancelarAluguer(1);

-- ====================================================
-- RM-006: Remover veículo
-- ====================================================
CALL RemoverVeiculo(1);

-- ====================================================
-- RM-007: Remover cliente
-- ====================================================
CALL RemoverCliente(1);

-- ====================================================
-- RM-008: Alterar estado do aluguer
-- ====================================================
CALL AlterarEstadoAluguer(2, 'Concluído');

-- ====================================================
-- RM-009: Veículos disponíveis num intervalo de tempo
-- ====================================================
CALL VeiculosDisponiveis('2025-06-01 10:00:00', '2025-06-05
↪  10:00:00');

-- ====================================================
-- RM-010: Veículos por características técnicas
-- ====================================================
CALL VeiculosPorCaracteristicas('SUV', 'Tesla', NULL, 'Elétrico',
↪  NULL);

-- ====================================================
-- RM-011: Histórico de aluguer do cliente
-- ====================================================
CALL HistoricoAluguerCliente(3);
```

```
-- ====================================================
-- RM-012: Obter cliente por número de telemóvel
-- ====================================================
CALL ClientePorTelemovel(912345678);

-- ====================================================
-- RM-013: Receita gerada por um posto
-- ====================================================
CALL ReceitaPosto(2, '2025-01-01', '2025-05-31');

-- ====================================================
-- RM-014: Taxa de ocupação de veículos de uma categoria
-- ====================================================
SELECT TaxaOcupacao('SUV', '2025-04-20 12:00:00') AS
↪  TaxaOcupacaoPercentual;

-- ====================================================
-- View: Ver todos os veículos com detalhes e posto
-- ====================================================
SELECT * FROM VeiculosComDetalhes WHERE Categoria = 'SUV';
```